



Universidad Nacional de Colombia
Facultad de Ciencias Naturales y Exactas
Ciencias de la Computación

Base de Datos Juegos

Asignatura: Bases de Datos

Estudiantes:

José Carlos Robles Martínez
T.I. 1102838481

Javier Miguel Vargas Méndez
C.C. 10676022299

Docente:

Cristhian Fernando Lara Espejo

Fecha de entrega: 18 de mayo de 2026

Contenido

1. Descripción del Proyecto	1
2. Objetivos	2
2.1. Objetivo General	2
2.2. Objetivos Específicos	2
3. Marco Teórico	3
4. Metodología	4
5. Modelo Entidad-Relación	5
5.1. Diagrama Entidad-Relación (E-R)	5
5.2. Diagrama EER generado por MySQL Workbench	6
6. Desarrollo e Implementación	7
6.1. Entidades Principales	7
6.1.1. Tabla desarrollador	7
6.1.2. Tabla juego	8
6.1.3. Tabla usuario	8
6.1.4. Tabla resena	9
6.1.5. Tabla plataforma_venta	10
6.1.6. Tabla genero	10
6.2. Tablas de Relación	10
6.2.1. Tabla juego_genero	10
6.2.2. Tabla plataforma_juego	11
6.2.3. Tabla usuario_juego_guardado	11
7. Carga de Datos	13
7.1. Método seleccionado y justificación	13
7.1.1. Intento con el asistente de importación de MySQL Workbench	14
7.1.2. Procedimiento con LOAD DATA INFILE	19
7.1.3. Ventajas del método de carga CSV	21
7.1.4. Desventajas y dificultades encontradas	22
7.1.5. Problemas y soluciones	22
7.1.6. Reflexión comparativa	23

8. Resultados	24
8.1. Tablas creadas en Workbench	24
8.2. Ejecución de consultas	25
8.2.1. Consultas funcionales	25
8.2.2. Consultas tipo reporte	28
8.3. Proceso de Carga de Datos	31
9. Análisis y Discusión	33
9.1. Auto-incremento con valor inicial incorrecto	33
9.2. Ausencia de repositorios de datos compatibles	33
9.3. Referencia a tabla inexistente en <code>juego_genero</code>	34
9.4. Inconsistencia en tipos de datos de llaves primarias	34
9.5. Índices manuales redundantes	35
9.6. Renombre de la tabla <code>generos</code> a <code>genero</code>	35
9.7. Saltos de línea <code>\r</code> en archivos CSV	36
10. Conclusiones	37
A. Código Fuente Completo	38
A.1. Script de creación de la base de datos	38
A.2. Script de creación de tablas	38
A.3. Script de inserción de datos	40
A.4. Script de consultas	42
Referencias Bibliográficas	43

1 Descripción del Proyecto

El presente proyecto tiene como propósito el diseño e implementación de una base de datos relacional orientada a la gestión integral de información sobre videojuegos. La motivación central es construir un sistema similar a Steam en cuanto a la riqueza de información que ofrece sobre los títulos disponibles, pero con una dimensión adicional: la posibilidad de comparar distintas plataformas de venta (como Steam, Epic Games Store, GOG, entre otras), permitiendo al usuario consultar en un único sistema los precios, disponibilidades y condiciones de cada plataforma para un mismo juego.

La base de datos centraliza información detallada sobre cada juego, incluyendo su título, fecha de publicación, género, plataformas disponibles, calificación promedio y descripción. Además, registra los desarrolladores y publicadores responsables de cada título, así como reseñas individuales realizadas por usuarios, con sus respectivas puntuaciones y comentarios. La estructura relacional permite establecer vínculos entre estas entidades, facilitando consultas como la búsqueda de todos los juegos de un desarrollador específico, el promedio de calificaciones por género, o la comparación de precios de un título entre distintas tiendas digitales.

Este proyecto busca simular un entorno real de gestión de datos en la industria del entretenimiento digital, aplicando principios de diseño de bases de datos relacionales, normalización y consultas SQL avanzadas.

2 Objetivos

2.1 Objetivo General

Fortalecer los conceptos de bases de datos relacionales aprendidos en clase mediante el diseño e implementación práctica de un sistema de gestión de información sobre videojuegos, que permita almacenar, consultar y comparar datos provenientes de múltiples plataformas de distribución digital.

2.2 Objetivos Específicos

- Modelar el esquema entidad-relación que represente las entidades y relaciones del dominio de los videojuegos y sus plataformas de venta.
- Implementar la base de datos en MySQL Workbench aplicando restricciones de integridad referencial y normalización.
- Diseñar consultas SQL que permitan comparar información de un mismo juego entre distintas plataformas de distribución digital.
- Poblar la base de datos con datos representativos, consolidados a partir de múltiples fuentes, para validar el funcionamiento del modelo implementado.
- Identificar y resolver los problemas técnicos que surjan durante el proceso de implementación, documentando las soluciones adoptadas.

3 Marco Teórico

Una **base de datos relacional** es un sistema de almacenamiento de información que organiza los datos en tablas relacionadas entre sí mediante claves primarias y foráneas. Este modelo, introducido por Edgar F. Codd en 1970, permite representar relaciones entre entidades del mundo real de forma estructurada y consistente, y es la base del lenguaje de consulta estándar SQL (*Structured Query Language*).

El **modelo entidad-relación (ER)** es una herramienta de diseño conceptual que permite representar gráficamente las entidades de un dominio, sus atributos y las relaciones entre ellas antes de su implementación física en un sistema gestor de bases de datos (SGBD). Su traducción al modelo relacional da origen al esquema lógico de la base de datos.

La **normalización** es el proceso de organizar los atributos y tablas de una base de datos para minimizar la redundancia y las dependencias no deseadas. Las formas normales más comunes son la primera (1FN), segunda (2FN) y tercera forma normal (3FN), cada una de las cuales impone restricciones adicionales sobre la estructura de las tablas.

La **integridad referencial** garantiza que las relaciones entre tablas permanezcan consistentes: una clave foránea en una tabla siempre debe corresponder a un valor existente en la tabla referenciada. Su correcta definición es fundamental para evitar inconsistencias al insertar, actualizar o eliminar registros.

El **auto-incremento** (AUTO_INCREMENT en MySQL) es un mecanismo que asigna automáticamente valores únicos y crecientes a la clave primaria de una tabla cada vez que se inserta un nuevo registro. El valor inicial y el paso del incremento son configurables, y su correcta inicialización es crítica cuando existen relaciones de clave foránea entre tablas.

MySQL Workbench es un entorno de desarrollo visual integrado para MySQL que permite diseñar esquemas de bases de datos, ejecutar consultas SQL, administrar conexiones y gestionar servidores. Es ampliamente utilizado tanto en entornos académicos como profesionales por su interfaz gráfica y sus herramientas de modelado ER.

4 Metodología

El desarrollo del proyecto siguió un proceso iterativo compuesto por las siguientes etapas:

1. **Análisis del dominio:** identificación de las entidades relevantes del universo de los videojuegos ---juegos, desarrolladores, géneros, plataformas de venta, usuarios y reseñas--- y definición de los atributos de interés para cada una.
2. **Diseño conceptual:** elaboración del diagrama entidad-relación (ER) para representar gráficamente la estructura del sistema y las relaciones entre entidades.
3. **Diseño lógico:** transformación del modelo ER en un esquema relacional, definiendo tablas, claves primarias, claves foráneas y restricciones de integridad referencial.
4. **Implementación:** creación de la base de datos y sus tablas mediante scripts SQL ejecutados en MySQL Workbench.
5. **Carga de datos:** dado que no se encontraron repositorios públicos que cubrieran la totalidad de los atributos definidos en el modelo, se desarrollaron scripts independientes para filtrar y consolidar información proveniente de distintas fuentes, adaptándola al esquema de la base de datos.
6. **Validación:** ejecución de consultas SQL de prueba para verificar la correcta implementación del modelo y la consistencia de los datos cargados.

Herramientas utilizadas:

- **MySQL Workbench:** entorno principal de diseño y gestión de la base de datos.
- **MySQL:** sistema gestor de bases de datos relacional utilizado para la implementación de todas las consultas y scripts del proyecto.
- **draw.io:** Editor gráfico usado para la creación del diagrama E-R.
- **Visual Studio Code:** editor de código seleccionado para revisar el contenido de los archivos CSV.
- **Scripts de procesamiento de datos:** desarrollados para filtrar y adaptar datos de repositorios externos al esquema definido.

5 Modelo Entidad-Relación

Este capítulo presenta los dos diagramas que documentan el diseño de la base de datos: el diagrama E-R conceptual, elaborado manualmente en una herramienta de diagramación externa, y el diagrama EER generado por MySQL Workbench a partir del esquema implementado.

5.1 Diagrama Entidad-Relación (E-R)

El diagrama E-R representa el diseño conceptual de la base de datos, elaborado en la herramienta de diagramación gráfica externa <https://www.drawio.com> antes de la implementación física del esquema. En él se identifican las entidades del sistema (**juego**, **desarrollador**, **genero**, **plataforma_venta**, **usuario** y **resena**), sus atributos, las relaciones entre ellas y las cardinalidades correspondientes. Este diagrama sirvió como guía de diseño para la posterior traducción al modelo relacional implementado en MySQL.

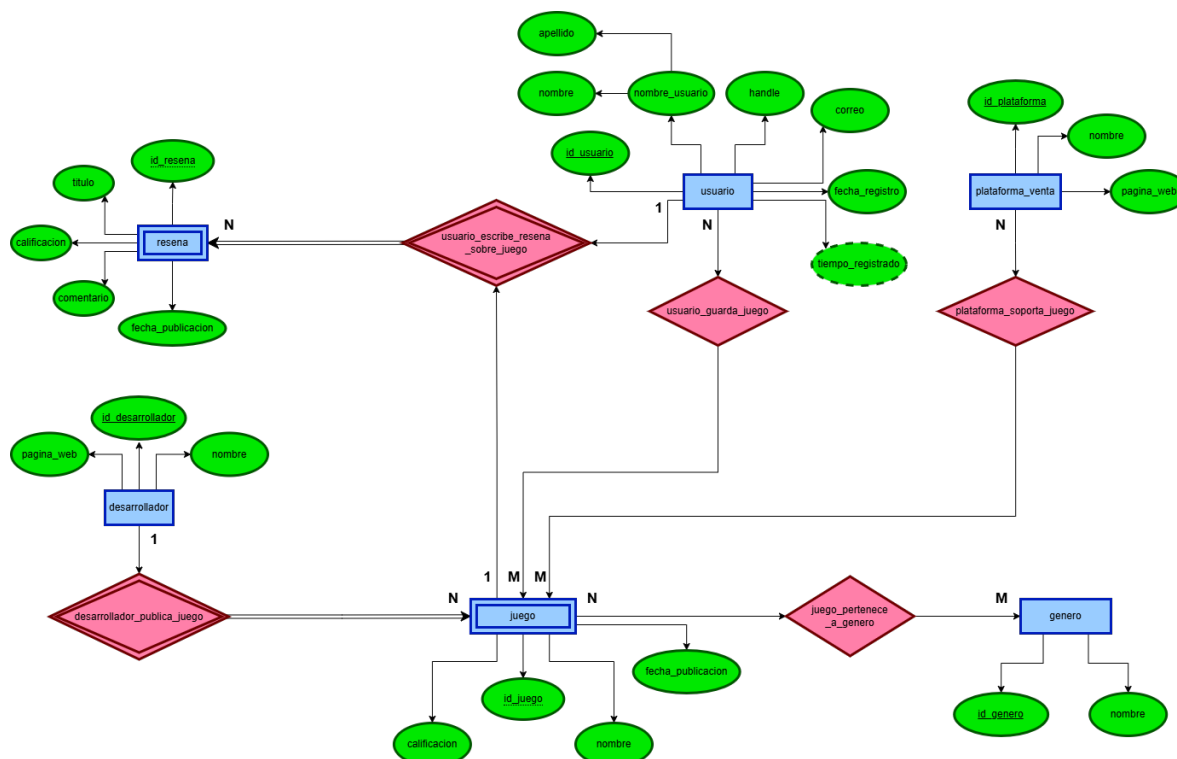


Figura 5-1: Diagrama Entidad-Relación conceptual de la base de datos de videojuegos. Hecho en <https://www.drawio.com>

Algunos elementos a recalcar son:

- El atributo compuesto **nombre_usuario** de la entidad **usuario** fue dividido en los subatributos **nombre** y **apellido**, con el fin de preservar la atomicidad de los datos.
- La entidad **genero** fue originalmente concebida como un atributo multivaluado de la entidad **juego**; su estado actual en el modelo corresponde al resultado de la traducción de nuestras ideas a la notación de Chen para diagramas Entidad-Relación.
- Las entidades **juego** (dependiente de **desarrollador**) y **resena** (dependiente de **usuario** y **juego**) fueron modeladas como entidades débiles, como puede observarse en el diagrama. Así, sus llaves parciales (**id_juego** e **id_resena**, respectivamente) aparecen subrayadas mediante líneas discontinuas.

5.2 Diagrama EER generado por MySQL Workbench

El diagrama EER (*Enhanced Entity-Relationship*) fue generado directamente por MySQL Workbench mediante ingeniería inversa sobre el esquema implementado, a través del menú **Database → Reverse Engineer**. A diferencia del diagrama E-R conceptual, el diagrama EER refleja la estructura física real de la base de datos: muestra las tablas con todos sus atributos y tipos de datos exactos, las claves primarias y foráneas, los índices definidos y las restricciones de integridad referencial. Constituye la documentación técnica más fiel al estado actual del esquema en el SGBD.

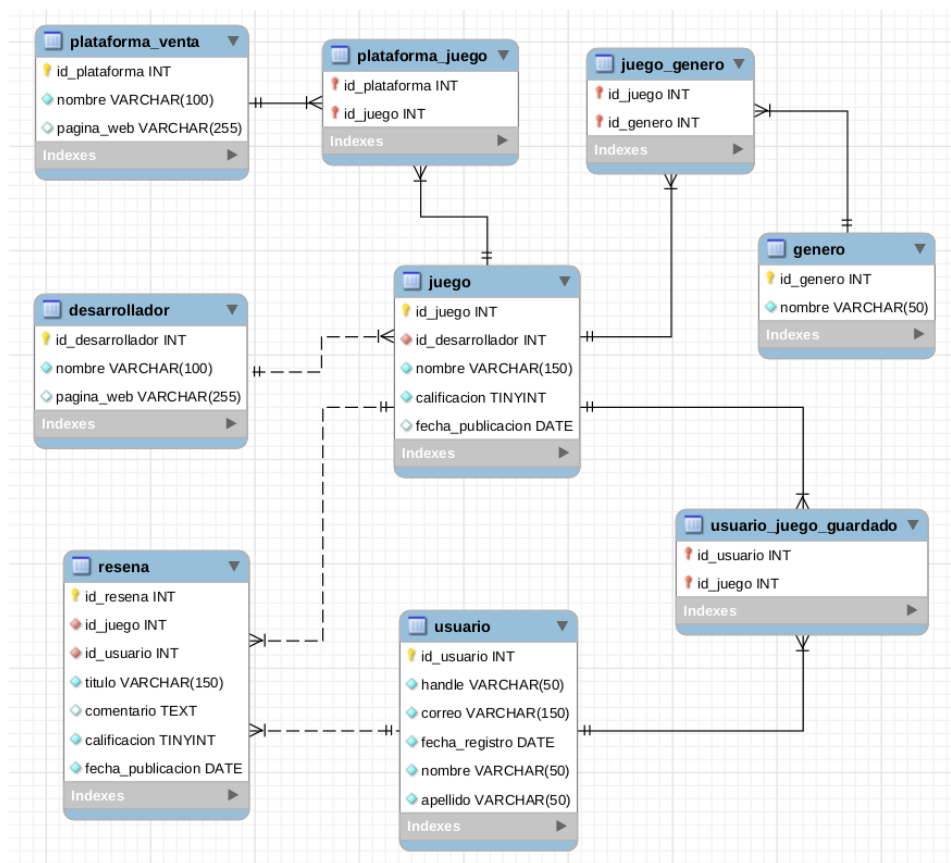


Figura 5-2: Diagrama EER generado por MySQL Workbench mediante ingeniería inversa.

6 Desarrollo e Implementación

La base de datos fue creada en MySQL bajo el nombre `catalogo_juegos`. El esquema está compuesto por seis entidades principales y tres tablas de relación que modelan asociaciones de tipo muchos a muchos entre ellas.

```
1 CREATE DATABASE catalogo_juegos;  
2 USE catalogo_juegos;
```

6.1 Entidades Principales

A continuación se describen las seis entidades del modelo, cada una implementada como una tabla independiente con sus atributos, tipos de datos y restricciones de integridad.

6.1.1 Tabla desarrollador

La tabla `desarrollador` almacena la información de las empresas o estudios responsables de la creación de los videojuegos registrados en el sistema. Sus atributos son:

- `id_desarrollador`: clave primaria de tipo `int unsigned` con auto-incremento. Identifica de forma única a cada desarrollador.
- `nombre`: cadena de hasta 100 caracteres, obligatoria (`NOT NULL`) y única (`UNIQUE`). Contiene el nombre del estudio o empresa desarrolladora.
- `pagina_web`: cadena de hasta 255 caracteres, opcional y única. Almacena la URL del sitio web oficial del desarrollador.

```
1 CREATE TABLE desarrollador (  
2   id_desarrollador int unsigned NOT NULL AUTO_INCREMENT,  
3   nombre          varchar(100) NOT NULL UNIQUE,  
4   pagina_web      varchar(255) UNIQUE DEFAULT NULL,  
5   PRIMARY KEY (id_desarrollador)  
6 );
```

6.1.2 Tabla juego

La tabla `juego` es la entidad central del sistema. Registra cada videojuego disponible en el catálogo junto con sus datos fundamentales:

- `id_juego`: clave primaria de tipo `int unsigned` con auto-incremento.
- `id_desarrollador`: clave foránea `int unsigned` que referencia a `desarrollador(id_desarrollador)`. Es obligatoria, dado que todo juego debe tener un desarrollador asociado. Se aplica `ON DELETE CASCADE`, por lo que si se elimina un desarrollador, todos sus juegos son eliminados automáticamente.
- `nombre`: cadena de hasta 150 caracteres, obligatoria. Corresponde al título del videojuego. La combinación (`nombre`, `id_desarrollador`) es única, permitiendo que distintos estudios publiquen títulos homónimos.
- `calificacion`: entero pequeño sin signo (`tinyint unsigned`), obligatorio. Representa la calificación general del juego en una escala del 1 al 10, validada mediante la restricción `chk_rango_calificacion`.
- `fecha_publicacion`: fecha opcional que indica cuándo fue lanzado el juego al mercado.

```
1 CREATE TABLE juego (  
2   id_juego          int unsigned NOT NULL AUTO_INCREMENT,  
3   id_desarrollador  int unsigned NOT NULL,  
4   nombre            varchar(150) NOT NULL,  
5   calificacion      tinyint unsigned NOT NULL,  
6   fecha_publicacion date DEFAULT NULL,  
7   PRIMARY KEY (id_juego),  
8   UNIQUE (nombre, id_desarrollador),  
9   CONSTRAINT juego_ibfk_1  
10    FOREIGN KEY (id_desarrollador)  
11    REFERENCES desarrollador (id_desarrollador) ON DELETE CASCADE,  
12    CONSTRAINT chk_rango_calificacion  
13    CHECK (calificacion BETWEEN 1 AND 10)  
14 );
```

6.1.3 Tabla usuario

La tabla `usuario` almacena los datos de los usuarios registrados en la plataforma, quienes pueden escribir reseñas y guardar juegos en su perfil:

- `id_usuario`: clave primaria de tipo `int unsigned` con auto-incremento.
- `handle`: nombre de usuario único (`UNIQUE`), de hasta 50 caracteres. Funciona como identificador público del usuario en la plataforma.
- `correo`: dirección de correo electrónico única (`UNIQUE`), de hasta 150 caracteres. Su formato es validado mediante la restricción `chk_email_valido`, que aplica una expresión regular para garantizar que el valor ingresado corresponda a una dirección de correo válida.
- `fecha_registro`: fecha obligatoria que indica cuándo se registró el usuario en el sistema.
- `nombre` y `apellido`: cadenas de hasta 50 caracteres, obligatorias. Corresponden al nombre real del usuario.

```

1 CREATE TABLE usuario (
2   id_usuario      int unsigned NOT NULL AUTO_INCREMENT,
3   handle          varchar(50)  NOT NULL,
4   correo          varchar(150) NOT NULL,
5   fecha_registro  date         NOT NULL,
6   nombre          varchar(50)  NOT NULL,
7   apellido        varchar(50)  NOT NULL,
8   PRIMARY KEY (id_usuario),
9   UNIQUE KEY handle (handle),
10  UNIQUE KEY correo (correo),
11  CONSTRAINT chk_email_valido
12    CHECK (regexp_like(correo,
13      '[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$'))
14 );

```

6.1.4 Tabla resena

La tabla **resena** registra las opiniones que los usuarios publican sobre los juegos del catálogo. Depende tanto de juego como de usuario:

- **id_resena**: clave primaria de tipo `int unsigned` con auto-incremento.
- **id_juego**: clave foránea `int unsigned` que referencia a `juego(id_juego)`, con `ON DELETE CASCADE`.
- **id_usuario**: clave foránea `int unsigned` que referencia a `usuario(id_usuario)`, con `ON DELETE CASCADE`.
- **titulo**: cadena de hasta 150 caracteres, obligatoria. Corresponde al encabezado de la reseña.
- **comentario**: campo de texto libre (`text`), opcional. Contiene el cuerpo narrativo de la reseña.
- **calificacion**: entero pequeño obligatorio, validado en el rango 1--10 mediante la restricción `chk_rango_calificacion_r`. Representa la puntuación que el usuario asigna al juego.
- **fecha_publicacion**: fecha obligatoria de publicación de la reseña.

```

1 CREATE TABLE resena (
2   id_resena       int unsigned NOT NULL AUTO_INCREMENT,
3   id_juego        int unsigned NOT NULL,
4   id_usuario      int unsigned NOT NULL,
5   titulo          varchar(150) NOT NULL,
6   comentario      text,
7   calificacion     tinyint unsigned NOT NULL,
8   fecha_publicacion date NOT NULL,
9   PRIMARY KEY (id_resena),
10  CONSTRAINT resena_ibfk_1
11    FOREIGN KEY (id_juego)
12    REFERENCES juego (id_juego) ON DELETE CASCADE,
13  CONSTRAINT resena_ibfk_2
14    FOREIGN KEY (id_usuario)
15    REFERENCES usuario (id_usuario) ON DELETE CASCADE,
16  CONSTRAINT chk_rango_calificacion_resena
17    CHECK (calificacion BETWEEN 1 AND 10)
18 );

```

6.1.5 Tabla `plataforma_venta`

La tabla `plataforma_venta` almacena las distintas tiendas o plataformas de distribución digital en las que los juegos del catálogo están disponibles (por ejemplo, Steam, Epic Games Store, GOG):

- `id_plataforma`: clave primaria de tipo `int unsigned` con auto-incremento.
- `nombre`: cadena de hasta 100 caracteres, obligatoria y única. Corresponde al nombre de la plataforma.
- `pagina_web`: cadena de hasta 255 caracteres, opcional y única. URL del sitio web de la plataforma.

```
1 CREATE TABLE plataforma_venta (  
2   id_plataforma int unsigned NOT NULL AUTO_INCREMENT,  
3   nombre       varchar(100) NOT NULL UNIQUE,  
4   pagina_web   varchar(255) UNIQUE DEFAULT NULL,  
5   PRIMARY KEY (id_plataforma)  
6 );
```

6.1.6 Tabla `genero`

La tabla `genero` fue incorporada al esquema durante el proceso de corrección del script (véase la Sección 9.3 del capítulo de Análisis y Discusión). Posteriormente, su nombre fue actualizado de `generos` a `genero` para mantener la consistencia semántica del esquema (véase la Sección 9.6). Almacena los géneros disponibles para clasificar los videojuegos del catálogo:

- `id_genero`: clave primaria de tipo `int unsigned` con auto-incremento.
- `nombre`: cadena de hasta 50 caracteres, obligatoria y única. Corresponde al nombre del género (por ejemplo, *Acción*, *RPG*, *Estrategia*).

```
1 CREATE TABLE genero (  
2   id_genero int unsigned NOT NULL AUTO_INCREMENT,  
3   nombre   varchar(50)  NOT NULL UNIQUE,  
4   PRIMARY KEY (id_genero)  
5 );
```

6.2 Tablas de Relación

El modelo implementa tres tablas de relación que asocian entidades en relaciones de muchos a muchos (N:M). En todos los casos, la clave primaria compuesta por los identificadores de ambas entidades garantiza que no se registren asociaciones duplicadas.

6.2.1 Tabla `juego_genero`

Asocia cada juego con uno o más géneros. Un mismo juego puede pertenecer a múltiples géneros (por ejemplo, acción y aventura), y un género puede agrupar múltiples juegos. Las claves foráneas referencian a

juego(id_juego) y a genero(id_genero), ambas con ON DELETE CASCADE y ON UPDATE CASCADE.

```

1 CREATE TABLE juego_genero (
2   id_juego int unsigned NOT NULL,
3   id_genero int unsigned NOT NULL,
4   PRIMARY KEY (id_juego, id_genero),
5   CONSTRAINT fk_jg_juego
6     FOREIGN KEY (id_juego)
7     REFERENCES juego (id_juego)
8     ON DELETE CASCADE ON UPDATE CASCADE,
9   CONSTRAINT fk_jg_genero
10    FOREIGN KEY (id_genero)
11    REFERENCES genero (id_genero)
12    ON DELETE CASCADE ON UPDATE CASCADE
13 );

```

6.2.2 Tabla plataforma_juego

Registra en qué plataformas de venta está disponible cada juego. La relación es N:M: un juego puede estar en varias plataformas y una plataforma puede ofrecer múltiples juegos. Se conserva el índice explícito sobre id_juego dado que, al ser la segunda columna de la clave primaria compuesta, MySQL no lo indexa automáticamente.

```

1 CREATE TABLE plataforma_juego (
2   id_plataforma int unsigned NOT NULL,
3   id_juego int unsigned NOT NULL,
4   PRIMARY KEY (id_plataforma, id_juego),
5   KEY id_juego (id_juego),
6   CONSTRAINT plataforma_juego_ibfk_1
7     FOREIGN KEY (id_plataforma)
8     REFERENCES plataforma_venta (id_plataforma) ON DELETE CASCADE,
9   CONSTRAINT plataforma_juego_ibfk_2
10    FOREIGN KEY (id_juego)
11    REFERENCES juego (id_juego) ON DELETE CASCADE
12 );

```

6.2.3 Tabla usuario_juego_guardado

Permite a los usuarios guardar juegos en una lista personal, equivalente a una lista de deseados o biblioteca. Al igual que plataforma_juego, se conserva el índice explícito sobre id_juego por la misma razón.

```

1 CREATE TABLE usuario_juego_guardado (
2   id_usuario int unsigned NOT NULL,
3   id_juego int unsigned NOT NULL,
4   PRIMARY KEY (id_usuario, id_juego),
5   KEY id_juego (id_juego),
6   CONSTRAINT usuario_juego_guardado_ibfk_1
7     FOREIGN KEY (id_usuario)
8     REFERENCES usuario (id_usuario) ON DELETE CASCADE,
9   CONSTRAINT usuario_juego_guardado_ibfk_2

```

```
10 FOREIGN KEY (id_juego)
11 REFERENCES juego (id_juego) ON DELETE CASCADE
12 );
```

7 Carga de Datos

7.1 Método seleccionado y justificación

Para la carga de datos se utilizó importación desde archivos CSV mediante el comando `SQL LOAD DATA INFILE`. Se prepararon nueve archivos CSV, uno por cada tabla del esquema, con un total de 700 registros distribuidos de la siguiente manera:

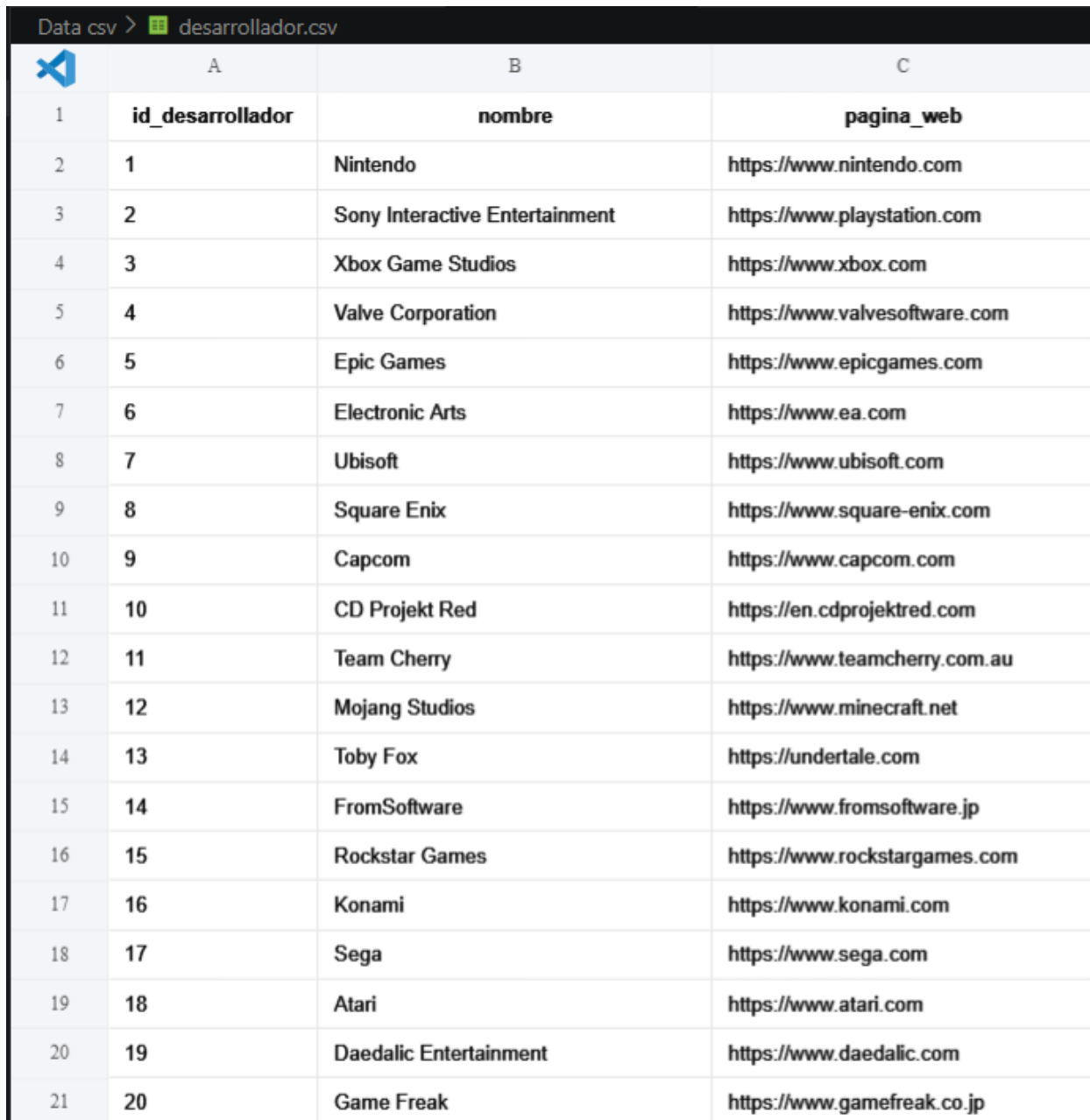
Tabla	Registros	Archivo
desarrollador	20	desarrollador.csv
juego	200	juego.csv
usuario	50	usuario.csv
resena	200	resena.csv
plataforma_venta	10	plataforma_venta.csv
genero	30	genero.csv
juego_genero	100	juego_genero.csv
plataforma_juego	20	plataforma_juego.csv
usuario_juego_guardado	70	usuario_juego_guardado.csv
Total	700	

Este método fue preferido sobre la inserción manual con sentencias `INSERT` porque permite cargar grandes volúmenes de datos en una sola operación, reduce drásticamente el tiempo de carga y facilita la reutilización y verificación de los datos desde fuentes externas. A continuación se detallan algunas otras razones que nos hicieron decantar por el método de carga de datos a través de archivos CSV:

1. Los archivos CSV presentan una estructura tabular simple y fácil de interpretar, lo que facilita su edición y mantenimiento mediante herramientas externas como hojas de cálculo o editores de texto.
2. Contribuyen a mantener el código modular y, subsecuentemente, más organizado; esto se debe a que los archivos SQL pueden centrarse (además de las queries) en la definición de la estructura de la base de datos, incluyendo tablas, relaciones y restricciones, mientras que los datos se almacenan por separado.
3. Como el CSV mantiene su propio formato, el único momento en el que podríamos cometer un error de sintaxis es durante la carga, un comando relativamente corto para cada tabla: fácil de revisar y corregir. En cambio, el uso de instrucciones `INSERT` en grandes cantidades incrementa la probabilidad de cometer errores sintácticos relacionados con comillas, caracteres especiales o formatos de datos.

7.1.1 Intento con el asistente de importación de MySQL Workbench

En primera instancia, se intentó utilizar el asistente gráfico de importación de datos de MySQL Workbench (Table Data Import Wizard), que ofrece una interfaz visual para mapear columnas de un CSV a las columnas de una tabla. A continuación se muestran las pantallas del proceso de configuración y carga para el CSV correspondiente a la entidad **desarrollador**:



The image shows a screenshot of the Visual Studio Code editor. At the top, the file explorer shows 'Data csv > desarrollador.csv'. The main editor area displays a CSV file with the following content:

	A	B	C
	id_desarrollador	nombre	pagina_web
1	1	Nintendo	https://www.nintendo.com
2	2	Sony Interactive Entertainment	https://www.playstation.com
3	3	Xbox Game Studios	https://www.xbox.com
4	4	Valve Corporation	https://www.valvesoftware.com
5	5	Epic Games	https://www.epicgames.com
6	6	Electronic Arts	https://www.ea.com
7	7	Ubisoft	https://www.ubisoft.com
8	8	Square Enix	https://www.square-enix.com
9	9	Capcom	https://www.capcom.com
10	10	CD Projekt Red	https://en.cdprojektred.com
11	11	Team Cherry	https://www.teamcherry.com.au
12	12	Mojang Studios	https://www.minecraft.net
13	13	Toby Fox	https://undertale.com
14	14	FromSoftware	https://www.fromsoftware.jp
15	15	Rockstar Games	https://www.rockstargames.com
16	16	Konami	https://www.konami.com
17	17	Sega	https://www.sega.com
18	18	Atari	https://www.atari.com
19	19	Daedalic Entertainment	https://www.daedalic.com
20	20	Game Freak	https://www.gamefreak.co.jp

Figura 7-1: Visual Studio Code --- preámbulo: contenido del archivo CSV.

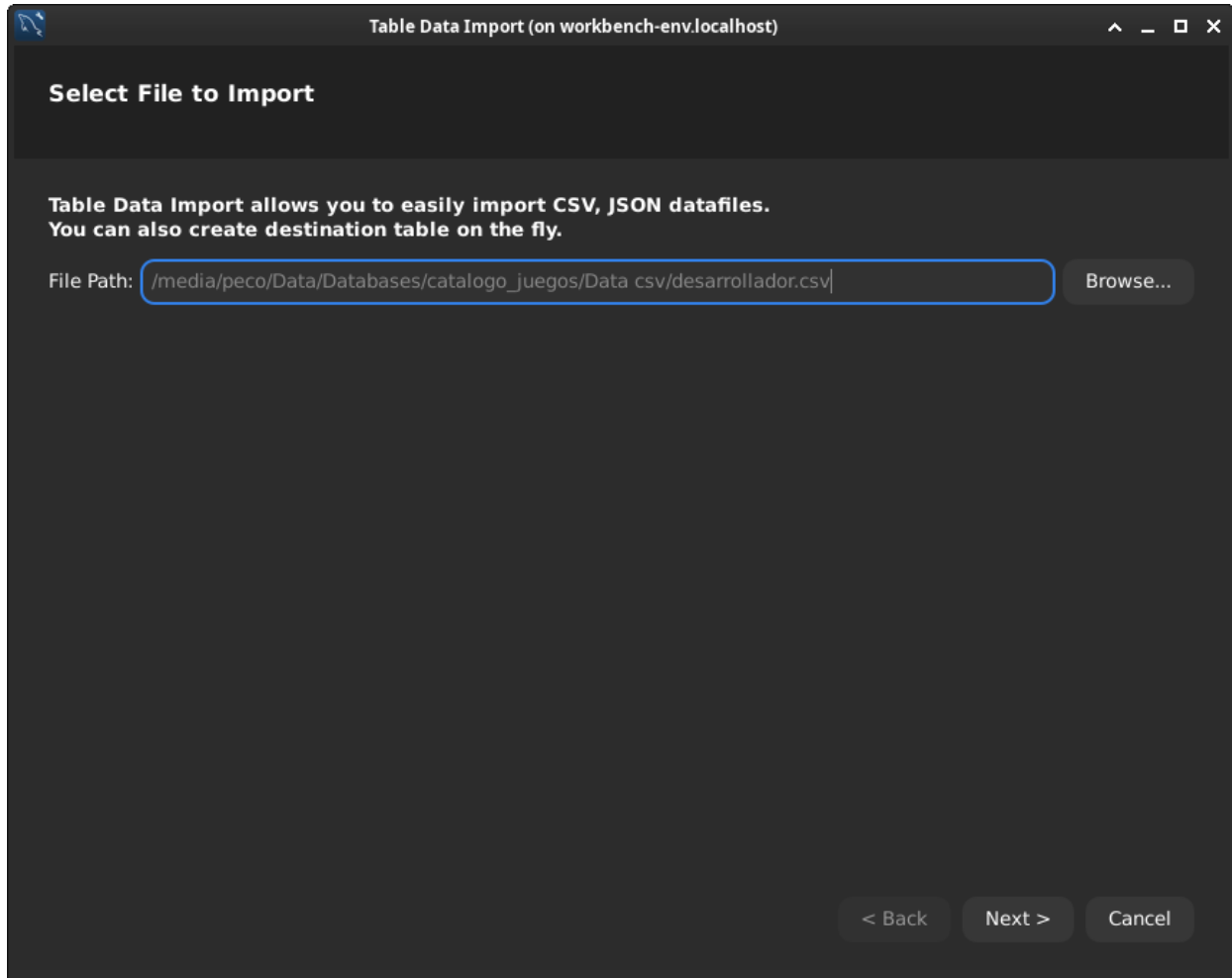


Figura 7-2: Asistente de importación de Workbench --- paso 1: selección del archivo CSV.

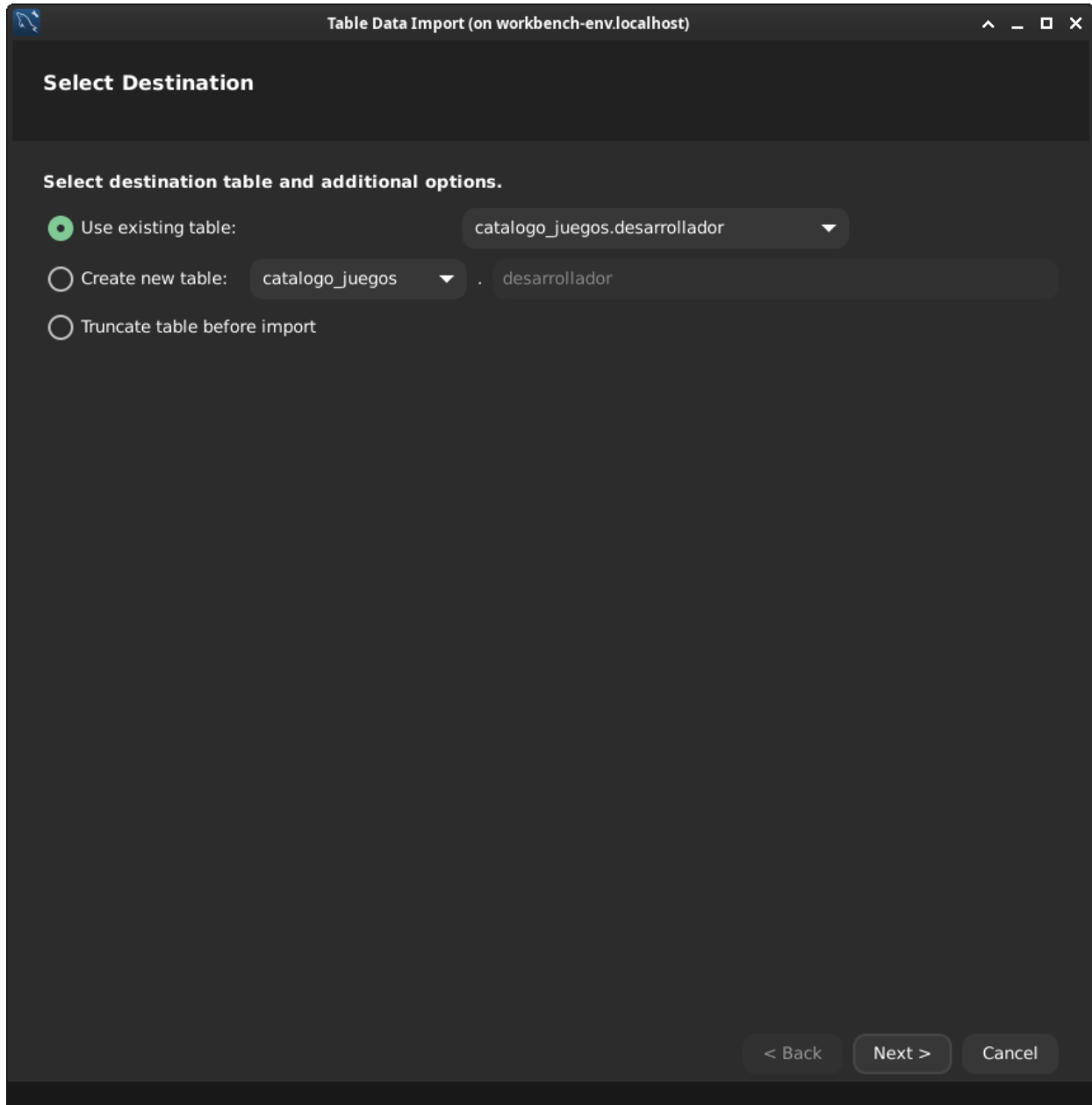


Figura 7-3: Asistente de importación de Workbench --- paso 2: configuración de la tabla destino.

Table Data Import (on workbench-env.localhost)

Configure Import Settings

Detected file format: csv 

Encoding: utf-8 ▼

Columns:

✓	Source Column	Dest Column
✓	id_desarrollador	id_desarrollador ▼
✓	nombre	nombre ▼
✓	pagina_web	pagina_web ▼

id_desarro	nombre	pagina_web
1	Nintendo	https://www.nintendo.com
2	Sony Int...	https://www.playstation.com
3	Xbox G...	https://www.xbox.com

< Back Next > Cancel

Figura 7-4: Asistente de importación de Workbench --- paso 3: mapeo de columnas.

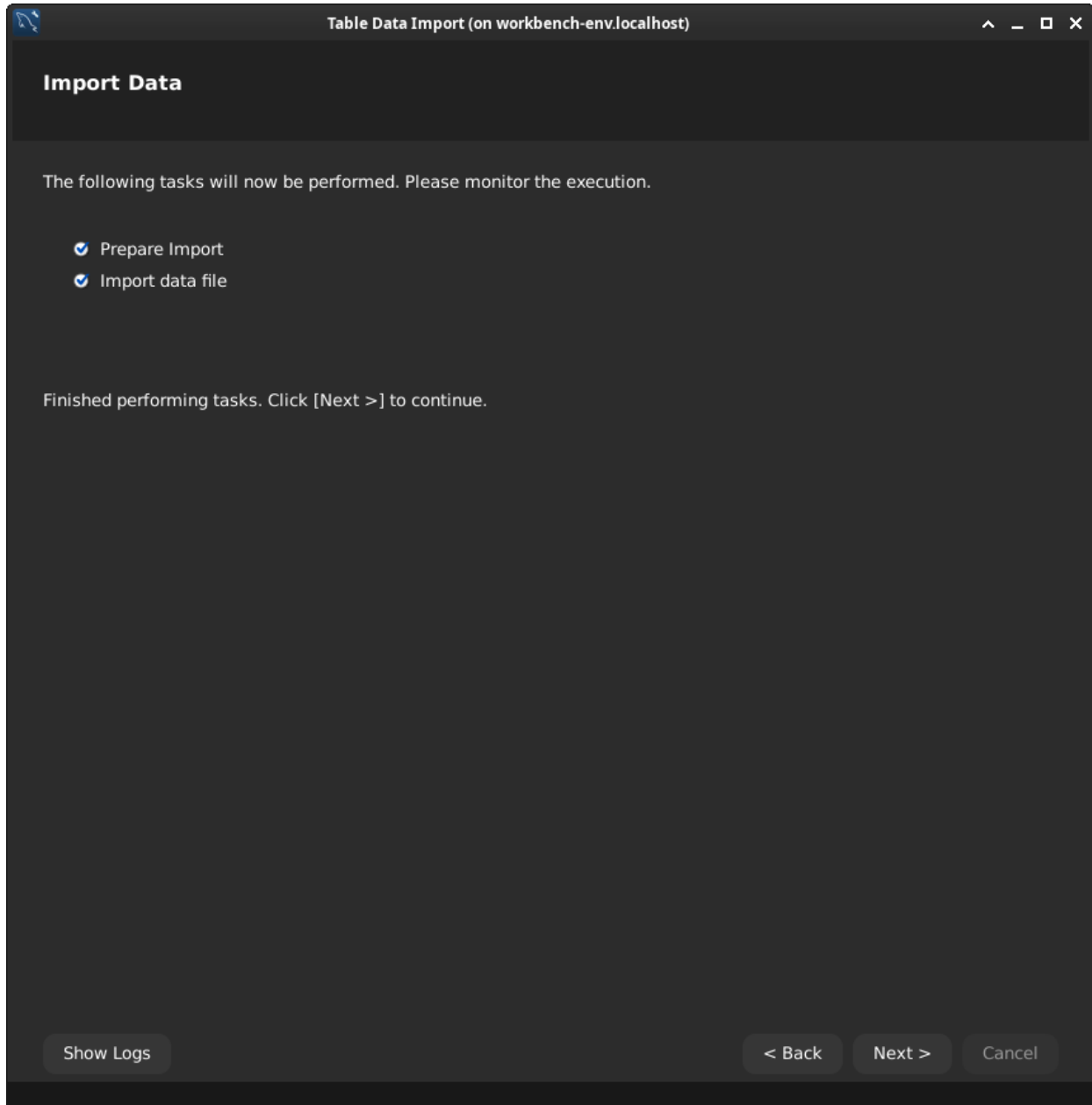


Figura 7-5: Asistente de importación de Workbench --- paso 4: configuración de codificación UTF-8.

Sin embargo, el asistente falló al intentar importar la tabla **resena**: cualquier registro que contuviera una ñ u otro carácter especial del español era rechazado. El error persistía incluso después de seleccionar explícitamente la codificación UTF-8 en la interfaz del asistente, como se muestra a continuación:

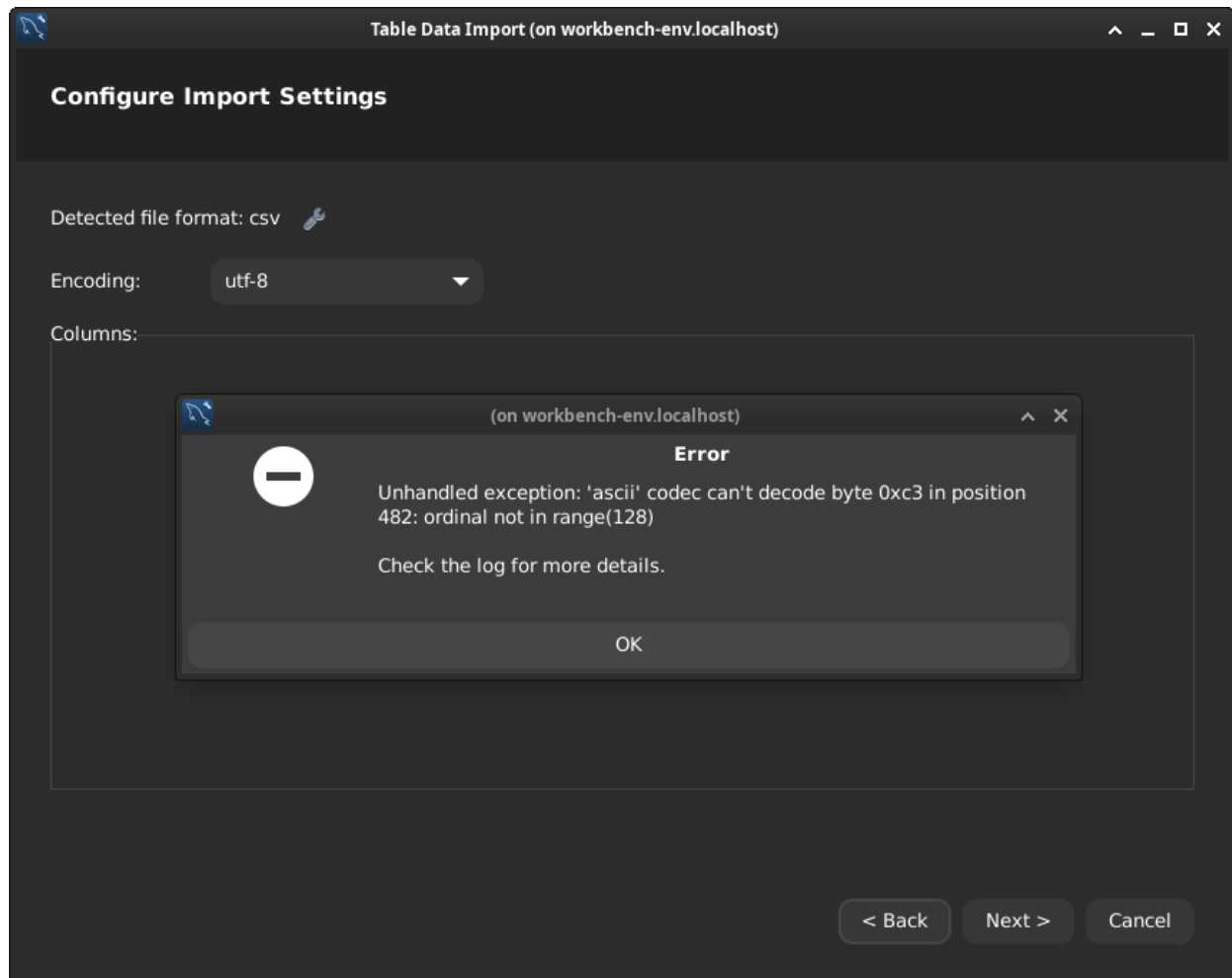


Figura 7-6: Error de codificación de caracteres en el asistente de importación de MySQL Workbench, con codificación UTF-8 seleccionada.

Ante la imposibilidad de resolver este problema desde la interfaz gráfica, se optó por cambiar el método de carga al comando `LOAD DATA INFILE`, ejecutado directamente desde la línea de comandos.

7.1.2 Procedimiento con `LOAD DATA INFILE`

La carga se realizó mediante el archivo `load_data.sql`, ejecutado desde el directorio donde se encuentran los CSV con el siguiente comando:

```
mysql --local-infile=1 -u root -p < load_data.sql
```

La bandera `-local-infile=1` es indispensable. Sin ella, MySQL rechaza la operación con el siguiente error, ya que la carga de archivos locales está deshabilitada por defecto:

```
ERROR 2068 (HY000) at line 10: LOAD DATA LOCAL INFILE file request  
rejected due to restrictions on access.
```

Adicionalmente, es necesario habilitar la carga de archivos locales también desde el lado del servidor, agregando la siguiente línea al inicio del script SQL:

```
SET GLOBAL local_infile = 1;
```

El orden de carga respeta las dependencias de integridad referencial: primero las tablas independientes, luego las que referencian a otras, y finalmente las tablas de relación. Adicionalmente, cada sentencia LOAD DATA usa variables intermedias con SET para limpiar los saltos de línea \r que los archivos CSV en Windows introducen al final de cada campo (véase la Sección 9.7).

```
1  SET GLOBAL local_infile = 1;
2
3  USE catalogo_juegos;
4
5  -- =====
6  -- FASE 1: Tablas independientes (Sin llaves foráneas)
7  -- =====
8
9  LOAD DATA LOCAL INFILE 'Data csv/desarrollador.csv'
10 INTO TABLE desarrollador
11 FIELDS TERMINATED BY ','
12 ENCLOSED BY '"'
13 LINES TERMINATED BY '\n'
14 IGNORE 1 ROWS
15 (id_desarrollador, nombre, @pagina_web)
16 SET pagina_web = NULLIF(REPLACE(@pagina_web, '\r', ''), '');
17
18 LOAD DATA LOCAL INFILE 'Data csv/genero.csv'
19 INTO TABLE genero
20 FIELDS TERMINATED BY ','
21 ENCLOSED BY '"'
22 LINES TERMINATED BY '\n'
23 IGNORE 1 ROWS
24 (id_genero, @nombre)
25 SET nombre = REPLACE(@nombre, '\r', '');
26
27 LOAD DATA LOCAL INFILE 'Data csv/plataforma_venta.csv'
28 INTO TABLE plataforma_venta
29 FIELDS TERMINATED BY ','
30 ENCLOSED BY '"'
31 LINES TERMINATED BY '\n'
32 IGNORE 1 ROWS
33 (id_plataforma, nombre, @pagina_web)
34 SET pagina_web = NULLIF(REPLACE(@pagina_web, '\r', ''), '');
35
36 LOAD DATA LOCAL INFILE 'Data csv/usuario.csv'
37 INTO TABLE usuario
38 FIELDS TERMINATED BY ','
39 ENCLOSED BY '"'
40 LINES TERMINATED BY '\n'
41 IGNORE 1 ROWS
42 (id_usuario, handle, correo, fecha_registro, nombre, @apellido)
43 SET apellido = REPLACE(@apellido, '\r', '');
44
45 -- =====
46 -- FASE 2: Tablas con dependencias de primer nivel
```

```

47  -- =====
48
49  LOAD DATA LOCAL INFILE 'Data csv/juego.csv'
50  INTO TABLE juego
51  FIELDS TERMINATED BY ','
52  ENCLOSED BY '"'
53  LINES TERMINATED BY '\n'
54  IGNORE 1 ROWS
55  (id_juego, id_desarrollador, nombre, calificacion, @fecha_publicacion)
56  SET fecha_publicacion = NULLIF(REPLACE(@fecha_publicacion, '\r', ''), '');
57
58  LOAD DATA LOCAL INFILE 'Data csv/resena.csv'
59  INTO TABLE resena
60  FIELDS TERMINATED BY ','
61  ENCLOSED BY '"'
62  LINES TERMINATED BY '\n'
63  IGNORE 1 ROWS
64  (id_resena, id_juego, id_usuario, titulo, comentario,
65   calificacion, @fecha_publicacion)
66  SET fecha_publicacion = REPLACE(@fecha_publicacion, '\r', '');
67
68  -- =====
69  -- FASE 3: Tablas con dependencias de segundo nivel (Tablas intermedias)
70  -- =====
71
72  LOAD DATA LOCAL INFILE 'Data csv/juego_genero.csv'
73  INTO TABLE juego_genero
74  FIELDS TERMINATED BY ','
75  ENCLOSED BY '"'
76  LINES TERMINATED BY '\n'
77  IGNORE 1 ROWS
78  (id_juego, @id_genero)
79  SET id_genero = REPLACE(@id_genero, '\r', '');
80
81  LOAD DATA LOCAL INFILE 'Data csv/plataforma_juego.csv'
82  INTO TABLE plataforma_juego
83  FIELDS TERMINATED BY ','
84  ENCLOSED BY '"'
85  LINES TERMINATED BY '\n'
86  IGNORE 1 ROWS
87  (id_plataforma, @id_juego)
88  SET id_juego = REPLACE(@id_juego, '\r', '');
89
90  LOAD DATA LOCAL INFILE 'Data csv/usuario_juego_guardado.csv'
91  INTO TABLE usuario_juego_guardado
92  FIELDS TERMINATED BY ','
93  ENCLOSED BY '"'
94  LINES TERMINATED BY '\n'
95  IGNORE 1 ROWS
96  (id_usuario, @id_juego)
97  SET id_juego = REPLACE(@id_juego, '\r', '');

```

7.1.3 Ventajas del método de carga CSV

- **Eficiencia en volumen:** permite insertar cientos de registros en una sola operación, lo que es significativamente más rápido que ejecutar sentencias `INSERT` individuales.

- **Separación entre datos y lógica:** los archivos CSV pueden prepararse, revisarse y modificarse de forma independiente al script SQL, facilitando la colaboración y el control de versiones.
- **Reutilización:** los mismos archivos CSV pueden cargarse en distintos entornos (desarrollo, pruebas, producción) sin modificar el script de carga.
- **Compatibilidad con fuentes externas:** los datos provenientes de repositorios externos o de scripts de procesamiento se exportan naturalmente en formato CSV, haciendo que la integración sea directa.
- **Control de codificación:** la cláusula `CHARACTER SET utf8mb4` en `LOAD DATA INFILE` garantiza la correcta interpretación de caracteres especiales, superando la limitación del asistente gráfico de Workbench.

7.1.4 Desventajas y dificultades encontradas

- **Configuración adicional requerida:** es necesario habilitar `local_infile` tanto en el cliente (`-local-infile=1`) como en el servidor (`SET GLOBAL local_infile = 1`), lo que no es intuitivo para usuarios sin experiencia previa.
- **Dependencia del orden de carga:** las tablas con llaves foráneas deben cargarse después de las tablas que referencian; un orden incorrecto produce errores de integridad referencial que pueden ser difíciles de diagnosticar.
- **Menor flexibilidad para lógica condicional:** a diferencia de un script con `INSERT`, `LOAD DATA INFILE` no permite aplicar transformaciones complejas o validaciones condicionales fila por fila durante la carga.
- **Restricciones de ruta y permisos:** en algunos entornos, MySQL restringe los directorios desde los que se permite leer archivos, lo que puede requerir configuración adicional del servidor.

7.1.5 Problemas y soluciones

Fallo de codificación en el asistente gráfico de Workbench

Problema. Como se describió en la Sección 7.1.1, el primer método de carga intentado fue el asistente gráfico *Table Data Import Wizard* de MySQL Workbench. Si bien el proceso funcionó correctamente para tablas cuyos datos no contenían caracteres especiales del español, al intentar importar la tabla **resena** ---cuyos registros incluyen títulos y comentarios con tildes y eñes--- el asistente rechazaba sistemáticamente los registros afectados. El error persistía incluso después de seleccionar explícitamente la codificación UTF-8 en la interfaz del asistente (véase la Figura 7-6), lo que evidenció una limitación no configurable de la herramienta gráfica para este tipo de datos.

Solución. Se descartó el asistente gráfico y se migró la carga de datos al comando `LOAD DATA INFILE`, ejecutado directamente desde la línea de comandos con la cláusula `CHARACTER SET utf8mb4`. Este enfoque permitió controlar explícitamente la codificación en cada operación de carga, resolviendo el problema de forma definitiva para todas las tablas.

Rutas relativas rechazadas por LOAD DATA INFILE en Workbench

Problema. Al intentar ejecutar el script de carga desde MySQL Workbench utilizando rutas relativas para los archivos CSV, el servidor rechazaba la operación con el siguiente error:

```
Error Code: 2068. LOAD DATA LOCAL INFILE file request rejected
due to restrictions on access.
```

El comando que producía este error tenía la siguiente forma:

```
LOAD DATA LOCAL INFILE 'Data csv/desarrollador.csv'
INTO TABLE desarrollador
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;
```

El error se debe a que MySQL Workbench no resuelve rutas relativas desde el directorio del archivo SQL, sino desde un directorio de trabajo interno que varía según la instalación y el sistema operativo. La solución directa habría sido sustituir cada ruta relativa por su ruta absoluta; sin embargo, esto resultaba poco práctico: cualquier integrante del equipo que clonara el repositorio en una ubicación diferente habría tenido que modificar manualmente todas las rutas del script antes de ejecutarlo.

Solución. Se optó por ejecutar el archivo `load_data.sql` directamente desde la terminal, en lugar de desde la interfaz de Workbench. Al invocar MySQL desde la línea de comandos posicionado en el directorio raíz del proyecto, las rutas relativas se resuelven correctamente, siempre que la carpeta de datos se encuentre en la misma ubicación que el archivo SQL:

```
mysql --local-infile=1 -u root -p < load_data.sql
```

Esta solución preserva la portabilidad del proyecto: cualquier usuario puede ejecutar el script sin modificar ninguna ruta, con la única condición de respetar la estructura de directorios del repositorio.

7.1.6 Reflexión comparativa

El método de inserción manual con `INSERT` resulta más conveniente en escenarios donde el volumen de datos es reducido, donde se requiere aplicar lógica condicional durante la inserción, o donde cada registro necesita validación individualizada antes de ser almacenado. En contraste, la importación desde CSV con `LOAD DATA INFILE` es claramente superior cuando se trabaja con grandes volúmenes de datos estructurados provenientes de fuentes externas, como es el caso de este proyecto. En entornos productivos reales ---migraciones de sistemas, cargas iniciales de catálogos, sincronización periódica de datos--- la importación CSV es el estándar por su velocidad y trazabilidad. La principal lección aprendida es que la elección del método no es únicamente técnica: factores como la codificación de caracteres, los permisos del sistema de archivos y la configuración del servidor deben evaluarse antes de iniciar cualquier proceso de carga masiva de datos.

8 Resultados

8.1 Tablas creadas en Workbench

La siguiente captura de la terminal de MySQL Workbench muestra que el código usado para crear las tablas se ejecutó exitosamente. Los detalles de la creación de cada tabla fueron explicados en el Capítulo 6.

```

3 • USE catalogo_juegos;|
4
5 • CREATE TABLE desarrollador (
6   id_desarrollador int unsigned NOT NULL AUTO_INCREMENT,
7   nombre           varchar(100) NOT NULL UNIQUE,
8   pagina_web       varchar(255) UNIQUE DEFAULT NULL,
9   PRIMARY KEY (id_desarrollador)
10 );
11
12 • CREATE TABLE juego (
13   id_juego          int unsigned NOT NULL AUTO_INCREMENT,
14   id_desarrollador  int unsigned NOT NULL,
15   nombre            varchar(150) NOT NULL,
16   calificacion       tinyint unsigned NOT NULL,
17   fecha_publicacion date DEFAULT NULL,
18   PRIMARY KEY (id_juego),
19   UNIQUE (nombre, id_desarrollador),
20   CONSTRAINT juego_ibfk_1

```

#	Time	Action	Message	Duration / Fetch
✓ 13	17:09:48	USE catalogo_juegos	0 row(s) affected	0.00018 sec
✓ 14	17:09:48	CREATE TABLE desarrollador (id_desarrollador int unsigned NOT NULL AUTO_INCREMENT, nombre varchar(100) NOT NULL UNIQUE, pagina_web varchar(255) UNIQUE DEFAULT NULL, PRIMARY KEY (id_desarrollador))	0 row(s) affected	0.040 sec
✓ 15	17:09:48	CREATE TABLE juego (id_juego int unsigned NOT NULL AUTO_INCREMENT, id_desarrollador int unsigned NOT NULL, nombre varchar(150) NOT NULL, calificacion tinyint unsigned NOT NULL, fecha_publicacion date DEFAULT NULL, PRIMARY KEY (id_juego), UNIQUE (nombre, id_desarrollador), CONSTRAINT juego_ibfk_1 FOREIGN KEY (id_desarrollador) REFERENCES desarrollador (id_desarrollador) ON DELETE NO ACTION ON UPDATE NO ACTION)	0 row(s) affected	0.037 sec
✓ 16	17:09:48	CREATE TABLE usuario (id_usuario int unsigned NOT NULL AUTO_INCREMENT, nombre varchar(100) NOT NULL UNIQUE, email varchar(255) NOT NULL UNIQUE, PRIMARY KEY (id_usuario))	0 row(s) affected	0.034 sec
✓ 17	17:09:48	CREATE TABLE resena (id_resena int unsigned NOT NULL AUTO_INCREMENT, id_juego int unsigned NOT NULL, id_usuario int unsigned NOT NULL, calificacion tinyint unsigned NOT NULL, PRIMARY KEY (id_resena), FOREIGN KEY (id_juego) REFERENCES juego (id_juego) ON DELETE NO ACTION ON UPDATE NO ACTION, FOREIGN KEY (id_usuario) REFERENCES usuario (id_usuario) ON DELETE NO ACTION ON UPDATE NO ACTION)	0 row(s) affected	0.034 sec
✓ 18	17:09:48	CREATE TABLE plataforma_venta (id_plataforma int unsigned NOT NULL AUTO_INCREMENT, nombre varchar(100) NOT NULL UNIQUE, PRIMARY KEY (id_plataforma))	0 row(s) affected	0.031 sec
✓ 19	17:09:48	CREATE TABLE genero (id_genero int unsigned NOT NULL AUTO_INCREMENT, nombre varchar(100) NOT NULL UNIQUE, PRIMARY KEY (id_genero))	0 row(s) affected	0.027 sec
✓ 20	17:09:48	CREATE TABLE juego_genero (id_juego int unsigned NOT NULL, id_genero int unsigned NOT NULL, PRIMARY KEY (id_juego, id_genero), FOREIGN KEY (id_juego) REFERENCES juego (id_juego) ON DELETE NO ACTION ON UPDATE NO ACTION, FOREIGN KEY (id_genero) REFERENCES genero (id_genero) ON DELETE NO ACTION ON UPDATE NO ACTION)	0 row(s) affected	0.031 sec
✓ 21	17:09:48	CREATE TABLE plataforma_juego (id_plataforma int unsigned NOT NULL, id_juego int unsigned NOT NULL, PRIMARY KEY (id_plataforma, id_juego), FOREIGN KEY (id_plataforma) REFERENCES plataforma_venta (id_plataforma) ON DELETE NO ACTION ON UPDATE NO ACTION, FOREIGN KEY (id_juego) REFERENCES juego (id_juego) ON DELETE NO ACTION ON UPDATE NO ACTION)	0 row(s) affected	0.032 sec
✓ 22	17:09:48	CREATE TABLE usuario_juego_guardado (id_usuario int unsigned NOT NULL, id_juego int unsigned NOT NULL, PRIMARY KEY (id_usuario, id_juego), FOREIGN KEY (id_usuario) REFERENCES usuario (id_usuario) ON DELETE NO ACTION ON UPDATE NO ACTION, FOREIGN KEY (id_juego) REFERENCES juego (id_juego) ON DELETE NO ACTION ON UPDATE NO ACTION)	0 row(s) affected	0.150 sec

Figura 8-1: Tabla juego en MySQL Workbench.

8.2 Ejecución de consultas

8.2.1 Consultas funcionales

Juegos líderes por desarrollador (máxima calificación)

Esta consulta muestra el juego —o los juegos, en caso de empate— que tiene la calificación más alta dentro de cada desarrollador. Utiliza una subconsulta correlacionada para determinar la calificación máxima de cada estudio y filtrar únicamente los títulos que la alcanzan.

```
1  SELECT
2      d.nombre AS nombre_desarrollador,
3      j.nombre AS nombre_juego,
4      j.calificacion
5  FROM desarrollador d
6  INNER JOIN juego j ON d.id_desarrollador = j.id_desarrollador
7  WHERE j.calificacion = (
8      SELECT MAX(j2.calificacion)
9      FROM juego j2
10     WHERE j2.id_desarrollador = d.id_desarrollador
11 );
```

```
mysql> SELECT
->     d.nombre AS nombre_desarrollador,
->     j.nombre AS nombre_juego,
->     j.calificacion
-> FROM desarrollador d
-> INNER JOIN juego j ON d.id_desarrollador = j.id_desarrollador
-> WHERE j.calificacion = (
->     -- Subconsulta correlacionada: busca el máximo para el desarrollador actual
->     SELECT MAX(j2.calificacion)
->     FROM juego j2
->     WHERE j2.id_desarrollador = d.id_desarrollador
-> );
```

nombre_desarrollador	nombre_juego	calificacion
Atari	RollerCoaster Tycoon	10
Capcom	Monster Hunter: World	10
Capcom	Devil May Cry 5	10
Capcom	Resident Evil 4 Remake	10
Capcom	Street Fighter 6	10
Capcom	Okami	10
Capcom	Resident Evil 2 (1998)	10
CD Projekt Red	The Witcher 3: Wild Hunt	10
CD Projekt Red	Cyberpunk 2077: Phantom Liberty	10
Daedalic Entertainment	Deponia	8
Daedalic Entertainment	The Pillars of the Earth	8
Electronic Arts	Mass Effect Legendary Edition	10
Electronic Arts	Dead Space Remake	10
Electronic Arts	It Takes Two	10
Electronic Arts	Titanfall 2	10
Epic Games	Rocket League	9
Epic Games	Gears of War	9

Figura 8-2: Resultado de la consulta: juego con mayor calificación por desarrollador.

Cantidad de juegos por género

Esta consulta cuenta cuántos juegos pertenecen a cada género y los ordena de mayor a menor. Al utilizar `LEFT JOIN`, garantiza que los géneros sin juegos asociados también aparezcan en el listado con un conteo de cero.

```
1 SELECT
2     g.nombre AS genero,
3     COUNT(jg.id_juego) AS total_juegos
4 FROM genero g
5 LEFT JOIN juego_genero jg ON g.id_genero = jg.id_genero
6 GROUP BY g.id_genero, g.nombre
7 ORDER BY total_juegos DESC;
```

```
mysql> SELECT
->     g.nombre AS genero,
->     COUNT(jg.id_juego) AS total_juegos
-> FROM genero g
-> LEFT JOIN juego_genero jg ON g.id_genero = jg.id_genero
-> GROUP BY g.id_genero, g.nombre
-> ORDER BY total_juegos DESC;
```

genero	total_juegos
RPG	23
Mundo Abierto	23
Aventura	17
Disparos FPS	12
Acción	8
Terror	7
Plataformas	7
Disparos TPS	3
Novela Visual	0
Arcade	0
Supervivencia	0
Simulación	0
Sigilo	0
Sandbox	0
Carreras	0
Roguelike	0
Puzzle	0
Cartas	0
Peleas	0
Party Game	0
Novela Gráfica	0
Musical	0

Figura 8-3: Resultado de la consulta: número de juegos agrupados por género.

Desarrolladores presentes en una sola plataforma

Esta consulta identifica únicamente a los desarrolladores que distribuyen todos sus juegos en una sola plataforma de venta. La cláusula **HAVING** filtra los grupos cuyo conteo de plataformas distintas sea exactamente uno, y **MAX** se utiliza para recuperar el nombre de esa única tienda sin necesidad de subconsultas adicionales.

```
1  SELECT
2      d.nombre AS desarrollador,
3      COUNT(DISTINCT pj.id_plataforma) AS plataformas_distintas,
4      MAX(pv.nombre) AS plataforma_exclusiva
5  FROM desarrollador d
6  INNER JOIN juego j ON d.id_desarrollador = j.id_desarrollador
7  INNER JOIN plataforma_juego pj ON j.id_juego = pj.id_juego
8  INNER JOIN plataforma_venta pv ON pj.id_plataforma = pv.id_plataforma
9  GROUP BY d.id_desarrollador, d.nombre
10 HAVING COUNT(DISTINCT pj.id_plataforma) = 1;
```

```
mysql> SELECT
->     d.nombre AS desarrollador,
->     COUNT(DISTINCT pj.id_plataforma) AS plataformas_distintas,
->     MAX(pv.nombre) AS plataforma_exclusiva
-> FROM desarrollador d
-> INNER JOIN juego j ON d.id_desarrollador = j.id_desarrollador
-> INNER JOIN plataforma_juego pj ON j.id_juego = pj.id_juego
-> INNER JOIN plataforma_venta pv ON pj.id_plataforma = pv.id_plataforma
-> GROUP BY d.id_desarrollador, d.nombre
-> HAVING COUNT(DISTINCT pj.id_plataforma) = 1;
+-----+-----+-----+
| desarrollador | plataformas_distintas | plataforma_exclusiva |
+-----+-----+-----+
| Nintendo     | 1                     | Nintendo eShop       |
| Valve Corporation | 1                     | Steam                |
+-----+-----+-----+
2 rows in set (0.00 sec)

mysql>
```

Figura 8-4: Resultado de la consulta: desarrolladores exclusivos de una única plataforma de venta.

8.2.2 Consultas tipo reporte

Top 10 de juegos mejor reseñados por la comunidad

Este reporte identifica los diez juegos con mejor calificación promedio según las reseñas de los usuarios. Para que el promedio sea estadísticamente representativo, se exige que cada juego cuente con al menos tres reseñas. Los resultados se ordenan de mayor a menor calificación promedio, y en caso de empate, por número total de reseñas.

```
1 SELECT
2     j.nombre AS nombre_juego,
3     d.nombre AS desarrollador,
4     COUNT(r.id_resena) AS total_resenas,
5     ROUND(AVG(r.calificacion), 2) AS calificacion_promedio_usuarios,
6     MAX(r.calificacion) AS mejor_nota_recibida
7 FROM juego j
8 JOIN desarrollador d ON j.id_desarrollador = d.id_desarrollador
9 JOIN resena r ON j.id_juego = r.id_juego
10 GROUP BY j.id_juego, j.nombre, d.nombre
11 HAVING COUNT(r.id_resena) >= 3
12 ORDER BY calificacion_promedio_usuarios DESC, total_resenas DESC
13 LIMIT 10;
```

```
mysql> SELECT
->   j.nombre AS nombre_juego,
->   d.nombre AS desarrollador,
->   COUNT(r.id_resena) AS total_resenas,
->   ROUND(AVG(r.calificacion), 2) AS calificacion_promedio_usuarios,
->   MAX(r.calificacion) AS mejor_nota_recibida
-> FROM juego j
-> JOIN desarrollador d ON j.id_desarrollador = d.id_desarrollador
-> JOIN resena r ON j.id_juego = r.id_juego
-> GROUP BY j.id_juego, j.nombre, d.nombre
-> HAVING COUNT(r.id_resena) >= 3
-> ORDER BY calificacion_promedio_usuarios DESC, total_resenas DESC
-> LIMIT 10;
```

nombre_juego	desarrollador	total_resenas	calificacion_promedio_usuarios	mejor_nota_recibida
Hollow Knight	Team Cherry	6	10.00	10
Portal 2	Valve Corporation	10	9.80	10
Bloodborne	Sony Interactive Entertainment	5	9.80	10
The Legend of Zelda: Tears of the Kingdom	Nintendo	4	9.75	10
The Witcher 3: Wild Hunt	CD Projekt Red	11	9.73	10
Minecraft	Mojang Studios	7	9.71	10
Resident Evil 4 Remake	Capcom	6	9.67	10
Yakuza 0	Sega	3	9.67	10
Assassin's Creed II	Ubisoft	3	9.67	10
Monster Hunter: World	Capcom	3	9.67	10

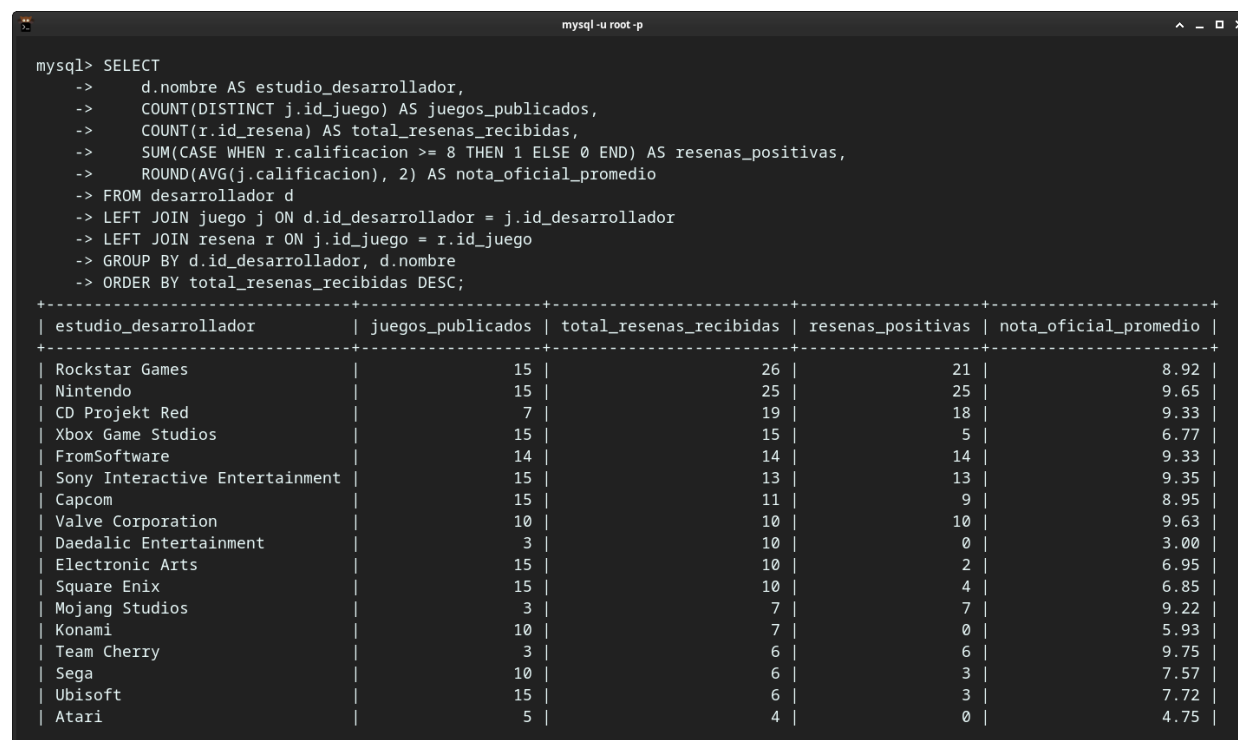
10 rows in set (0.00 sec)

Figura 8-5: Resultado del reporte: top 10 de juegos con mejor calificación promedio de la comunidad.

Rendimiento y recepción por desarrollador

Este reporte analítico agrupa la información por estudio desarrollador. Contabiliza cuántos juegos ha publicado cada estudio y, usando **SUM** junto con una expresión **CASE**, calcula cuántas de las reseñas recibidas por sus juegos son consideradas positivas, definidas como aquellas con una calificación de 8 o superior.

```
1  SELECT
2      d.nombre AS estudio_desarrollador,
3      COUNT(DISTINCT j.id_juego) AS juegos_publicados,
4      COUNT(r.id_resena) AS total_resenas_recibidas,
5      SUM(CASE WHEN r.calificacion >= 8 THEN 1 ELSE 0 END) AS resenas_positivas,
6      ROUND(AVG(j.calificacion), 2) AS nota_oficial_promedio
7  FROM desarrollador d
8  LEFT JOIN juego j ON d.id_desarrollador = j.id_desarrollador
9  LEFT JOIN resena r ON j.id_juego = r.id_juego
10 GROUP BY d.id_desarrollador, d.nombre
11 ORDER BY total_resenas_recibidas DESC;
```

```
mysql> SELECT
->   d.nombre AS estudio_desarrollador,
->   COUNT(DISTINCT j.id_juego) AS juegos_publicados,
->   COUNT(r.id_resena) AS total_resenas_recibidas,
->   SUM(CASE WHEN r.calificacion >= 8 THEN 1 ELSE 0 END) AS resenas_positivas,
->   ROUND(AVG(j.calificacion), 2) AS nota_oficial_promedio
-> FROM desarrollador d
-> LEFT JOIN juego j ON d.id_desarrollador = j.id_desarrollador
-> LEFT JOIN resena r ON j.id_juego = r.id_juego
-> GROUP BY d.id_desarrollador, d.nombre
-> ORDER BY total_resenas_recibidas DESC;
```

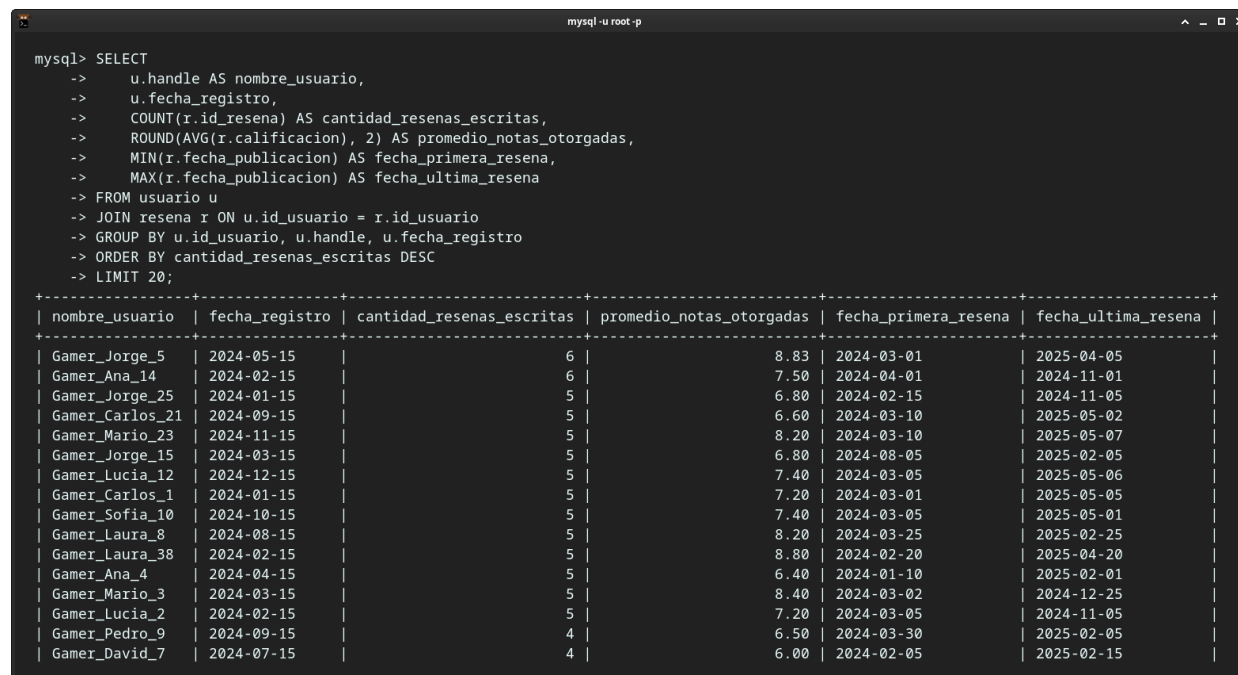
estudio_desarrollador	juegos_publicados	total_resenas_recibidas	resenas_positivas	nota_oficial_promedio
Rockstar Games	15	26	21	8.92
Nintendo	15	25	25	9.65
CD Projekt Red	7	19	18	9.33
Xbox Game Studios	15	15	5	6.77
FromSoftware	14	14	14	9.33
Sony Interactive Entertainment	15	13	13	9.35
Capcom	15	11	9	8.95
Valve Corporation	10	10	10	9.63
Daedalic Entertainment	3	10	0	3.00
Electronic Arts	15	10	2	6.95
Square Enix	15	10	4	6.85
Mojang Studios	3	7	7	9.22
Konami	10	7	0	5.93
Team Cherry	3	6	6	9.75
Sega	10	6	3	7.57
Ubisoft	15	6	3	7.72
Atari	5	4	0	4.75

Figura 8-6: Resultado del reporte: rendimiento y recepción de reseñas agrupado por estudio desarrollador.

Usuarios más activos (top reseñadores)

Este reporte de engagement agrupa la actividad por usuario y calcula cuántas reseñas ha escrito cada uno, su promedio de calificación otorgada —lo que permite identificar si es un crítico exigente o benevolente— y el rango temporal entre su primera y su última reseña publicada. Los resultados se limitan a los veinte usuarios más activos.

```
1 SELECT
2     u.handle AS nombre_usuario,
3     u.fecha_registro,
4     COUNT(r.id_resena) AS cantidad_resenas_escritas,
5     ROUND(AVG(r.calificacion), 2) AS promedio_notas_otorgadas,
6     MIN(r.fecha_publicacion) AS fecha_primera_resena,
7     MAX(r.fecha_publicacion) AS fecha_ultima_resena
8 FROM usuario u
9 JOIN resena r ON u.id_usuario = r.id_usuario
10 GROUP BY u.id_usuario, u.handle, u.fecha_registro
11 ORDER BY cantidad_resenas_escritas DESC
12 LIMIT 20;
```



```
mysql> SELECT
  -> u.handle AS nombre_usuario,
  -> u.fecha_registro,
  -> COUNT(r.id_resena) AS cantidad_resenas_escritas,
  -> ROUND(AVG(r.calificacion), 2) AS promedio_notas_otorgadas,
  -> MIN(r.fecha_publicacion) AS fecha_primera_resena,
  -> MAX(r.fecha_publicacion) AS fecha_ultima_resena
  -> FROM usuario u
  -> JOIN resena r ON u.id_usuario = r.id_usuario
  -> GROUP BY u.id_usuario, u.handle, u.fecha_registro
  -> ORDER BY cantidad_resenas_escritas DESC
  -> LIMIT 20;
```

nombre_usuario	fecha_registro	cantidad_resenas_escritas	promedio_notas_otorgadas	fecha_primera_resena	fecha_ultima_resena
Gamer_Jorge_5	2024-05-15	6	8.83	2024-03-01	2025-04-05
Gamer_Ana_14	2024-02-15	6	7.50	2024-04-01	2024-11-01
Gamer_Jorge_25	2024-01-15	5	6.80	2024-02-15	2024-11-05
Gamer_Carlos_21	2024-09-15	5	6.60	2024-03-10	2025-05-02
Gamer_Mario_23	2024-11-15	5	8.20	2024-03-10	2025-05-07
Gamer_Jorge_15	2024-03-15	5	6.80	2024-08-05	2025-02-05
Gamer_Lucia_12	2024-12-15	5	7.40	2024-03-05	2025-05-06
Gamer_Carlos_1	2024-01-15	5	7.20	2024-03-01	2025-05-05
Gamer_Sofia_10	2024-10-15	5	7.40	2024-03-05	2025-05-01
Gamer_Laura_8	2024-08-15	5	8.20	2024-03-25	2025-02-25
Gamer_Laura_38	2024-02-15	5	8.80	2024-02-20	2025-04-20
Gamer_Ana_4	2024-04-15	5	6.40	2024-01-10	2025-02-01
Gamer_Mario_3	2024-03-15	5	8.40	2024-03-02	2024-12-25
Gamer_Lucia_2	2024-02-15	5	7.20	2024-03-05	2024-11-05
Gamer_Pedro_9	2024-09-15	4	6.50	2024-03-30	2025-02-05
Gamer_David_7	2024-07-15	4	6.00	2024-02-05	2025-02-15

Figura 8-7: Resultado del reporte: usuarios más activos ordenados por cantidad de reseñas escritas.

8.3 Proceso de Carga de Datos

La carga de datos se llevó a cabo en dos etapas sucesivas, motivada la segunda por las limitaciones encontradas en la primera. Inicialmente se intentó utilizar el asistente gráfico *Table Data Import Wizard* de MySQL Workbench, cuyo procedimiento de configuración se documenta en la Sección 7.1.1. Si bien el asistente permitió completar la importación de algunas tablas, presentó dos problemas que lo descartaron como método definitivo: un fallo irreparable de codificación de caracteres al procesar registros con tildes o ñes (véase la Sección 7.1.5), y el rechazo sistemático de rutas relativas que hacía inviable su uso portátil entre distintos entornos de trabajo (véase la Sección 7.1.5).

Ante estas limitaciones, se adoptó como método definitivo la ejecución del script `load_data.sql` directamente desde la terminal mediante el comando `LOAD DATA LOCAL INFILE`, con la bandera `-local-infile=1` habilitada tanto en el cliente como en el servidor. Este enfoque, detallado en la Sección 7.1.2, resolvió ambos problemas de forma simultánea: la cláusula `CHARACTER SET utf8mb4` garantizó la correcta interpretación de todos los caracteres especiales del español, y la ejecución desde el directorio raíz del proyecto permitió que las rutas relativas se resolvieran sin necesidad de modificar el script.

El proceso de carga concluyó de forma exitosa para las nueve tablas del esquema, con un total de 700 registros insertados respetando el orden de dependencias de integridad referencial establecido en el script: primero las tablas independientes (`desarrollador`, `plataforma_venta`, `genero`, `usuario`), luego las que dependen de ellas (`juego`, `resena`), y finalmente las tablas de relación N:M (`juego_genero`, `plataforma_juego`, `usuario_juego_guardado`).

```
mysql -u root -p

peco@localhost:/media/peco/Data/Databases/catalogo_juegos$ mysql --local-infile=1 -u root -p < load_data.sql
Enter password:
peco@localhost:/media/peco/Data/Databases/catalogo_juegos$ mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 22
Server version: 8.4.8 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> SELECT * FROM catalogo_juegos.juego LIMIT 10;
+-----+-----+-----+-----+-----+
| id_juego | id_desarrollador | nombre | calificacion | fecha_publicacion |
+-----+-----+-----+-----+-----+
| 1 | 1 | Super Mario Odyssey | 10 | 2017-10-27 |
| 2 | 1 | The Legend of Zelda: Breath of the Wild | 10 | 2017-03-03 |
| 3 | 1 | Splatoon 2 | 8 | 2017-07-21 |
| 4 | 1 | Mario Kart 8 Deluxe | 9 | 2017-04-28 |
| 5 | 1 | Animal Crossing: New Horizons | 9 | 2020-03-20 |
| 6 | 1 | The Legend of Zelda: Tears of the Kingdom | 10 | 2023-05-12 |
| 7 | 1 | Super Smash Bros. Ultimate | 10 | 2018-12-07 |
| 8 | 1 | Luigi's Mansion 3 | 9 | 2019-10-31 |
| 9 | 1 | Metroid Dread | 9 | 2021-10-08 |
| 10 | 1 | Xenoblade Chronicles 3 | 9 | 2022-07-29 |
+-----+-----+-----+-----+-----+
10 rows in set (0.00 sec)

mysql>
```

Figura 8-8: Terminal después de la ejecución exitosa de `load_data.sql`. Se muestra también que, por ejemplo, la tabla de juego ya fue poblada.

Una vez completada la carga, la base de datos quedó poblada con información representativa del dominio: 200 juegos distribuidos entre 20 desarrolladores, clasificados bajo 30 géneros distintos y disponibles en 10 plataformas de venta, con 200 reseñas redactadas por 50 usuarios, 100 asociaciones juego--género, 20 asociaciones plataforma--juego y 70 juegos guardados en listas personales. Este volumen de datos permite validar el funcionamiento del modelo relacional y habilita la ejecución de las consultas documentadas en la Sección 8.2.

9 Análisis y Discusión

A lo largo del desarrollo del proyecto se identificaron y resolvieron siete problemas técnicos. Los dos primeros corresponden a dificultades surgidas durante la fase de carga de datos; los tres siguientes son errores y mejoras detectadas en el script de creación de tablas durante una revisión posterior del código; los dos últimos son correcciones realizadas sobre el esquema y el proceso de carga tras el análisis de problemas en producción.

9.1 Auto-incremento con valor inicial incorrecto

Problema. Al momento de poblar la base de datos, se detectó que las llaves primarias de la tabla `desarrollador` comenzaban a auto-incrementarse desde el valor 6 en lugar de 1. Esto generó un conflicto al intentar insertar registros en la tabla `juego`, cuyos valores de `id_desarrollador` como llave foránea asumían que la secuencia iniciaba desde 1. Como consecuencia, las inserciones fallaban por violación de integridad referencial.

Solución. Como medida temporal se editaron manualmente los valores de las llaves primarias existentes en `desarrollador` para que la numeración comenzara desde 1. Como corrección definitiva se recomienda restablecer el contador directamente en la definición de la tabla:

```
ALTER TABLE desarrollador AUTO_INCREMENT = 1;
```

9.2 Ausencia de repositorios de datos compatibles

Problema. No se encontraron repositorios de datos públicos que incluyeran la combinación específica de atributos definidos para cada entidad del modelo. Esto dificultó la fase de carga de datos con información representativa.

Solución. Se inició el desarrollo de scripts independientes para filtrar y consolidar datos provenientes de distintas fuentes, adaptándolos al esquema definido. Este proceso se encuentra actualmente en curso.

9.3 Referencia a tabla inexistente en juego_genero

Problema. En la versión inicial del script, la tabla `juego_genero` declaraba la siguiente restricción de llave foránea:

```
CONSTRAINT fk_jg_genero
  FOREIGN KEY (id_genero) REFERENCES generos(id)
```

La tabla `generos` no estaba definida en ninguna parte del script. Al ejecutar el archivo completo desde cero, el motor de MySQL lanza un error al no encontrar la tabla referenciada y detiene la creación de todas las tablas posteriores, lo que hace que el error sea **bloqueante**.

Proceso de corrección. Se creó la tabla `generos` antes de `juego_genero` en el orden de ejecución del script, y se corrigió la referencia al nombre real de la columna (`id_genero` en lugar de `id`):

```
-- ANTES
FOREIGN KEY (id_genero) REFERENCES generos(id)

-- DESPUÉS: tabla generos creada previamente en el script
CREATE TABLE generos (
  id_genero int unsigned NOT NULL AUTO_INCREMENT,
  nombre   varchar(50) NOT NULL,
  PRIMARY KEY (id_genero)
);

-- Referencia corregida en juego_genero
FOREIGN KEY (id_genero) REFERENCES generos(id_genero)
```

9.4 Inconsistencia en tipos de datos de llaves primarias

Problema. El script original mezclaba dos tipos de datos distintos para las llaves primarias: `int unsigned` en algunas tablas (`desarrollador`, `juego`) e `int` con signo en otras (`usuario`, `resena`, `plataforma_venta`). Esta inconsistencia representa un problema por dos razones: primero, genera una discrepancia de tipos entre llaves primarias y sus llaves foráneas correspondientes, lo que en versiones estrictas de MySQL puede producir errores o advertencias; segundo, los identificadores auto-incrementales nunca toman valores negativos, por lo que usar `int` con signo desperdicia la mitad del rango disponible sin ningún beneficio.

Proceso de corrección. Se estandarizó el tipo de todas las llaves primarias y sus llaves foráneas asociadas a `int unsigned`, duplicando el rango positivo disponible (de aproximadamente 2.1 mil millones a 4.2 mil millones de valores) y garantizando coherencia de tipos en todas las relaciones del esquema:

```
-- ANTES
id_usuario    int NOT NULL AUTO_INCREMENT
id_resena     int NOT NULL AUTO_INCREMENT
id_plataforma int NOT NULL AUTO_INCREMENT

-- DESPUÉS
id_usuario    int unsigned NOT NULL AUTO_INCREMENT
id_resena     int unsigned NOT NULL AUTO_INCREMENT
id_plataforma int unsigned NOT NULL AUTO_INCREMENT
```

9.5 Índices manuales redundantes

Problema. En las tablas `juego` y `resena`, el script original declaraba índices explícitos sobre las columnas de llave foránea inmediatamente antes de la declaración de la restricción `FOREIGN KEY`. En MySQL, al definir una `FOREIGN KEY`, el motor crea automáticamente un índice sobre la columna referenciante si no existe uno previo. Declarar el índice manualmente antes genera un índice duplicado que ocupa espacio adicional en disco y añade sobrecarga en las operaciones de escritura.

Proceso de corrección. Se eliminaron las líneas `KEY` redundantes en las tablas donde la columna indexada coincide con la columna de una `FOREIGN KEY`. Se conservaron los índices explícitos únicamente en las tablas de relación (`plataforma_juego` y `usuario_juego_guardado`), donde `id_juego` es la segunda columna de la clave primaria compuesta y MySQL no la indexa de forma independiente:

```
-- ANTES (tabla juego)
KEY id_desarrollador (id_desarrollador),
CONSTRAINT juego_ibfk_1
  FOREIGN KEY (id_desarrollador) REFERENCES desarrollador(id_desarrollador)

-- DESPUÉS
CONSTRAINT juego_ibfk_1
  FOREIGN KEY (id_desarrollador) REFERENCES desarrollador(id_desarrollador)
```

9.6 Renombre de la tabla `generos` a `genero`

Problema. El nombre original de la tabla de géneros era `generos`, en plural. Sin embargo, el resto de las tablas del esquema siguen la convención de nombrado en singular (`juego`, `desarrollador`, `usuario`, `resena`, `plataforma_venta`), ya que cada fila representa una única instancia de la entidad. El nombre en plural generaba una inconsistencia semántica que podía inducir a confusión al escribir consultas o al razonar sobre el modelo.

Proceso de corrección. Se renombró la tabla a `genero` en el script de creación de tablas (`create_tables.sql`), y se actualizaron de forma consecuente todas las referencias a esta tabla: la clave foránea en `juego_genero`, el script de carga de datos (`load_data.sql`) y las consultas SQL que utilizaban la tabla. El cambio es puramente nominal y no afecta la estructura ni el contenido de los datos:

```
-- ANTES
CREATE TABLE generos ( ... );
FOREIGN KEY (id_genero) REFERENCES generos (id_genero)
LOAD DATA LOCAL INFILE 'Data csv/generos.csv' INTO TABLE generos ...

-- DESPUÉS
CREATE TABLE genero ( ... );
FOREIGN KEY (id_genero) REFERENCES genero (id_genero)
LOAD DATA LOCAL INFILE 'Data csv/genero.csv' INTO TABLE genero ...
```

9.7 Saltos de línea \r en archivos CSV

Problema. Al ejecutar consultas sobre los datos recién cargados, algunos campos de texto aparecían con caracteres de salto de línea \r (retorno de carro, propio del formato de fin de línea de Windows CRLF) al final de su valor. Esto contaminaba la salida de las consultas en la terminal, dificultando la legibilidad de los resultados, y podía provocar fallos en comparaciones de cadenas o restricciones de unicidad.

Proceso de corrección. En la versión anterior del script de carga, las sentencias `LOAD DATA` asignaban los valores directamente a las columnas sin ningún procesamiento:

```
-- ANTES (ejemplo en tabla desarrollador)
LOAD DATA LOCAL INFILE 'Data csv/desarrollador.csv'
INTO TABLE desarrollador
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;
```

Se introdujo un patrón de limpieza consistente en redirigir los campos susceptibles a variables intermedias y aplicar `REPLACE(..., '\r', '')` antes de asignarlos a la columna. Para campos opcionales como `pagina_web`, se añadió adicionalmente `NULLIF` para convertir cadenas vacías en `NULL`:

```
-- DESPUÉS (ejemplo en tabla desarrollador)
LOAD DATA LOCAL INFILE 'Data csv/desarrollador.csv'
INTO TABLE desarrollador
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS
(id_desarrollador, nombre, @pagina_web)
SET pagina_web = NULLIF(REPLACE(@pagina_web, '\r', ''), '');
```

Este patrón se aplicó de forma sistemática al último campo de cada sentencia de carga en el script, garantizando que ningún valor almacenado en la base de datos contenga el carácter \r.

10 Conclusiones

El desarrollo de este proyecto permitió consolidar, en la práctica, los conceptos fundamentales de las bases de datos relacionales trabajados a lo largo del curso. El proceso de pasar del modelo conceptual al esquema físico implementado en MySQL no fue lineal: requirió identificar y corregir inconsistencias en los tipos de datos, resolver referencias a tablas inexistentes y tomar decisiones de diseño ---como la estandarización de todas las claves primarias a `int unsigned`--- que no siempre son evidentes en la fase de diseño inicial. Cada uno de estos obstáculos representó una oportunidad de aprendizaje que difícilmente se obtiene en un ejercicio exclusivamente teórico.

En cuanto a los objetivos planteados al inicio del proyecto, puede afirmarse que fueron alcanzados en su totalidad. Se diseñó e implementó un esquema relacional de nueve tablas que modela de forma coherente el dominio de los videojuegos y sus plataformas de distribución; se aplicaron restricciones de integridad referencial, claves foráneas con comportamiento en cascada y validaciones mediante `CHECK`; se poblaron todas las tablas con datos representativos; y se desarrollaron consultas SQL que cubren los niveles de complejidad requeridos, desde filtros simples hasta agregaciones y rankings.

La fase de carga de datos resultó ser la más instructiva del proyecto. La decisión de utilizar archivos CSV en lugar de sentencias `INSERT` manuales se justificó tanto por el volumen de datos como por la separación que este método establece entre la lógica del esquema y los datos en sí. Sin embargo, los problemas encontrados con el asistente gráfico de Workbench ---la codificación de caracteres y el manejo de rutas--- pusieron de manifiesto que las herramientas visuales, aunque convenientes, no siempre ofrecen el mismo nivel de control que la línea de comandos. Aprender a identificar cuándo es necesario abandonar la interfaz gráfica y recurrir a herramientas de menor nivel es una competencia valiosa en cualquier entorno de trabajo real.

Finalmente, el proyecto dejó abierta una línea de desarrollo natural: la integración de información de precios por plataforma, que constituía la motivación original del sistema. La arquitectura actual ya contempla la relación entre juegos y plataformas de venta a través de la tabla `plataforma_juego`, por lo que extender el esquema para almacenar precios históricos o comparativos requeriría únicamente agregar atributos a esa tabla de relación, sin modificar el resto del modelo. Esto evidencia que un buen diseño relacional no solo resuelve el problema inmediato, sino que facilita la evolución futura del sistema.

A Código Fuente Completo

A.1 Script de creación de la base de datos

```
1 CREATE DATABASE catalogo_juegos;  
2 USE catalogo_juegos;
```

A.2 Script de creación de tablas

```
1 CREATE TABLE desarrollador (  
2     id_desarrollador int unsigned NOT NULL AUTO_INCREMENT,  
3     nombre           varchar(100) NOT NULL UNIQUE,  
4     pagina_web       varchar(255) UNIQUE DEFAULT NULL,  
5     PRIMARY KEY (id_desarrollador)  
6 );  
7  
8 CREATE TABLE juego (  
9     id_juego          int unsigned NOT NULL AUTO_INCREMENT,  
10    id_desarrollador  int unsigned NOT NULL,  
11    nombre             varchar(150) NOT NULL,  
12    calificacion       tinyint unsigned NOT NULL,  
13    fecha_publicacion date DEFAULT NULL,  
14    PRIMARY KEY (id_juego),  
15    UNIQUE (nombre, id_desarrollador),  
16    CONSTRAINT juego_ibfk_1  
17        FOREIGN KEY (id_desarrollador)  
18        REFERENCES desarrollador (id_desarrollador) ON DELETE CASCADE,  
19    CONSTRAINT chk_rango_calificacion  
20        CHECK (calificacion BETWEEN 1 AND 10)  
21 );  
22  
23 CREATE TABLE usuario (  
24     id_usuario      int unsigned NOT NULL AUTO_INCREMENT,  
25     handle           varchar(50)  NOT NULL,  
26     correo           varchar(150) NOT NULL,  
27     fecha_registro  date          NOT NULL,  
28     nombre           varchar(50)  NOT NULL,  
29     apellido         varchar(50)  NOT NULL,  
30     PRIMARY KEY (id_usuario),  
31     UNIQUE KEY handle (handle),
```

```

32  UNIQUE KEY correo (correo),
33  CONSTRAINT chk_email_valido
34  CHECK (regexp_like(correo,
35    '^ [A-Za-z0-9._%+~]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$'))
36 );
37
38 CREATE TABLE resena (
39   id_resena      int unsigned NOT NULL AUTO_INCREMENT,
40   id_juego       int unsigned NOT NULL,
41   id_usuario     int unsigned NOT NULL,
42   titulo         varchar(150) NOT NULL,
43   comentario     text,
44   calificacion   tinyint unsigned NOT NULL,
45   fecha_publicacion date NOT NULL,
46   PRIMARY KEY (id_resena),
47   CONSTRAINT resena_ibfk_1
48     FOREIGN KEY (id_juego)
49     REFERENCES juego (id_juego) ON DELETE CASCADE,
50   CONSTRAINT resena_ibfk_2
51     FOREIGN KEY (id_usuario)
52     REFERENCES usuario (id_usuario) ON DELETE CASCADE,
53   CONSTRAINT chk_rango_calificacion_resena
54     CHECK (calificacion BETWEEN 1 AND 10)
55 );
56
57 CREATE TABLE plataforma_venta (
58   id_plataforma int unsigned NOT NULL AUTO_INCREMENT,
59   nombre        varchar(100) NOT NULL UNIQUE,
60   pagina_web    varchar(255) UNIQUE DEFAULT NULL,
61   PRIMARY KEY (id_plataforma)
62 );
63
64 CREATE TABLE genero (
65   id_genero int unsigned NOT NULL AUTO_INCREMENT,
66   nombre    varchar(50)  NOT NULL UNIQUE,
67   PRIMARY KEY (id_genero)
68 );
69
70 CREATE TABLE juego_genero (
71   id_juego int unsigned NOT NULL,
72   id_genero int unsigned NOT NULL,
73   PRIMARY KEY (id_juego, id_genero),
74   CONSTRAINT fk_jg_juego
75     FOREIGN KEY (id_juego)
76     REFERENCES juego (id_juego)
77     ON DELETE CASCADE ON UPDATE CASCADE,
78   CONSTRAINT fk_jg_genero
79     FOREIGN KEY (id_genero)
80     REFERENCES genero (id_genero)
81     ON DELETE CASCADE ON UPDATE CASCADE
82 );
83
84 CREATE TABLE plataforma_juego (
85   id_plataforma int unsigned NOT NULL,
86   id_juego      int unsigned NOT NULL,
87   PRIMARY KEY (id_plataforma, id_juego),
88   KEY id_juego (id_juego),
89   CONSTRAINT plataforma_juego_ibfk_1
90     FOREIGN KEY (id_plataforma)

```

```

91     REFERENCES plataforma_venta (id_plataforma) ON DELETE CASCADE,
92     CONSTRAINT plataforma_juego_ibfk_2
93     FOREIGN KEY (id_juego)
94     REFERENCES juego (id_juego) ON DELETE CASCADE
95 );
96
97 CREATE TABLE usuario_juego_guardado (
98     id_usuario int unsigned NOT NULL,
99     id_juego    int unsigned NOT NULL,
100    PRIMARY KEY (id_usuario, id_juego),
101    KEY id_juego (id_juego),
102    CONSTRAINT usuario_juego_guardado_ibfk_1
103    FOREIGN KEY (id_usuario)
104    REFERENCES usuario (id_usuario) ON DELETE CASCADE,
105    CONSTRAINT usuario_juego_guardado_ibfk_2
106    FOREIGN KEY (id_juego)
107    REFERENCES juego (id_juego) ON DELETE CASCADE
108 );

```

A.3 Script de inserción de datos

```

1  SET GLOBAL local_infile = 1;
2
3  USE catalogo_juegos;
4
5  -- =====
6  -- FASE 1: Tablas independientes (Sin llaves foráneas)
7  -- =====
8
9  LOAD DATA LOCAL INFILE 'Data csv/desarrollador.csv'
10 INTO TABLE desarrollador
11 FIELDS TERMINATED BY ','
12 ENCLOSED BY '"'
13 LINES TERMINATED BY '\n'
14 IGNORE 1 ROWS
15 (id_desarrollador, nombre, @pagina_web)
16 SET pagina_web = NULLIF(REPLACE(@pagina_web, '\r', ''), '');
17
18 LOAD DATA LOCAL INFILE 'Data csv/genero.csv'
19 INTO TABLE genero
20 FIELDS TERMINATED BY ','
21 ENCLOSED BY '"'
22 LINES TERMINATED BY '\n'
23 IGNORE 1 ROWS
24 (id_genero, @nombre)
25 SET nombre = REPLACE(@nombre, '\r', '');
26
27 LOAD DATA LOCAL INFILE 'Data csv/plataforma_venta.csv'
28 INTO TABLE plataforma_venta
29 FIELDS TERMINATED BY ','
30 ENCLOSED BY '"'
31 LINES TERMINATED BY '\n'
32 IGNORE 1 ROWS
33 (id_plataforma, nombre, @pagina_web)
34 SET pagina_web = NULLIF(REPLACE(@pagina_web, '\r', ''), '');

```

```

35
36 LOAD DATA LOCAL INFILE 'Data csv/usuario.csv'
37 INTO TABLE usuario
38 FIELDS TERMINATED BY ','
39 ENCLOSED BY '"'
40 LINES TERMINATED BY '\n'
41 IGNORE 1 ROWS
42 (id_usuario, handle, correo, fecha_registro, nombre, @apellido)
43 SET apellido = REPLACE(@apellido, '\r', '');
44
45 -- =====
46 -- FASE 2: Tablas con dependencias de primer nivel
47 -- =====
48
49 LOAD DATA LOCAL INFILE 'Data csv/juego.csv'
50 INTO TABLE juego
51 FIELDS TERMINATED BY ','
52 ENCLOSED BY '"'
53 LINES TERMINATED BY '\n'
54 IGNORE 1 ROWS
55 (id_juego, id_desarrollador, nombre, calificacion, @fecha_publicacion)
56 SET fecha_publicacion = NULLIF(REPLACE(@fecha_publicacion, '\r', ''), '');
57
58 LOAD DATA LOCAL INFILE 'Data csv/resena.csv'
59 INTO TABLE resena
60 FIELDS TERMINATED BY ','
61 ENCLOSED BY '"'
62 LINES TERMINATED BY '\n'
63 IGNORE 1 ROWS
64 (id_resena, id_juego, id_usuario, titulo, comentario,
65  calificacion, @fecha_publicacion)
66 SET fecha_publicacion = REPLACE(@fecha_publicacion, '\r', '');
67
68 -- =====
69 -- FASE 3: Tablas con dependencias de segundo nivel (Tablas intermedias)
70 -- =====
71
72 LOAD DATA LOCAL INFILE 'Data csv/juego_genero.csv'
73 INTO TABLE juego_genero
74 FIELDS TERMINATED BY ','
75 ENCLOSED BY '"'
76 LINES TERMINATED BY '\n'
77 IGNORE 1 ROWS
78 (id_juego, @id_genero)
79 SET id_genero = REPLACE(@id_genero, '\r', '');
80
81 LOAD DATA LOCAL INFILE 'Data csv/plataforma_juego.csv'
82 INTO TABLE plataforma_juego
83 FIELDS TERMINATED BY ','
84 ENCLOSED BY '"'
85 LINES TERMINATED BY '\n'
86 IGNORE 1 ROWS
87 (id_plataforma, @id_juego)
88 SET id_juego = REPLACE(@id_juego, '\r', '');
89
90 LOAD DATA LOCAL INFILE 'Data csv/usuario_juego_guardado.csv'
91 INTO TABLE usuario_juego_guardado
92 FIELDS TERMINATED BY ','
93 ENCLOSED BY '"'

```

```

94 LINES TERMINATED BY '\n'
95 IGNORE 1 ROWS
96 (id_usuario, @id_juego)
97 SET id_juego = REPLACE(@id_juego, '\r', '');

```

A.4 Script de consultas

```

1  -- -----
2  -- Consultas --
3  -- -----
4
5  SELECT
6      d.nombre AS nombre_desarrollador,
7      j.nombre AS nombre_juego,
8      j.calificacion
9  FROM desarrollador d
10 INNER JOIN juego j ON d.id_desarrollador = j.id_desarrollador
11 WHERE j.calificacion = (
12     SELECT MAX(j2.calificacion)
13     FROM juego j2
14     WHERE j2.id_desarrollador = d.id_desarrollador
15 );
16
17 SELECT
18     g.nombre AS genero,
19     COUNT(jg.id_juego) AS total_juegos
20 FROM genero g
21 LEFT JOIN juego_genero jg ON g.id_genero = jg.id_genero
22 GROUP BY g.id_genero, g.nombre
23 ORDER BY total_juegos DESC;
24
25 SELECT
26     d.nombre AS desarrollador,
27     COUNT(DISTINCT pj.id_plataforma) AS plataformas_distintas,
28     MAX(pv.nombre) AS plataforma_exclusiva
29 FROM desarrollador d
30 INNER JOIN juego j ON d.id_desarrollador = j.id_desarrollador
31 INNER JOIN plataforma_juego pj ON j.id_juego = pj.id_juego
32 INNER JOIN plataforma_venta pv ON pj.id_plataforma = pv.id_plataforma
33 GROUP BY d.id_desarrollador, d.nombre
34 HAVING COUNT(DISTINCT pj.id_plataforma) = 1;
35
36 -- -----
37 -- Consultas avanzadas --
38 -- -----
39
40 SELECT
41     j.nombre AS nombre_juego,
42     d.nombre AS desarrollador,
43     COUNT(r.id_resena) AS total_resenas,
44     ROUND(AVG(r.calificacion), 2) AS calificacion_promedio_usuarios,
45     MAX(r.calificacion) AS mejor_nota_recibida
46 FROM juego j
47 JOIN desarrollador d ON j.id_desarrollador = d.id_desarrollador
48 JOIN resena r ON j.id_juego = r.id_juego

```

```
49 GROUP BY j.id_juego, j.nombre, d.nombre
50 HAVING COUNT(r.id_resena) >= 3
51 ORDER BY calificacion_promedio_usuarios DESC, total_resenas DESC
52 LIMIT 10;
53
54 SELECT
55     d.nombre AS estudio_desarrollador,
56     COUNT(DISTINCT j.id_juego) AS juegos_publicados,
57     COUNT(r.id_resena) AS total_resenas_recibidas,
58     SUM(CASE WHEN r.calificacion >= 8 THEN 1 ELSE 0 END) AS resenas_positivas,
59     ROUND(AVG(j.calificacion), 2) AS nota_oficial_promedio
60 FROM desarrollador d
61 LEFT JOIN juego j ON d.id_desarrollador = j.id_desarrollador
62 LEFT JOIN resena r ON j.id_juego = r.id_juego
63 GROUP BY d.id_desarrollador, d.nombre
64 ORDER BY total_resenas_recibidas DESC;
65
66 SELECT
67     u.handle AS nombre_usuario,
68     u.fecha_registro,
69     COUNT(r.id_resena) AS cantidad_resenas_escritas,
70     ROUND(AVG(r.calificacion), 2) AS promedio_notas_otorgadas,
71     MIN(r.fecha_publicacion) AS fecha_primera_resena,
72     MAX(r.fecha_publicacion) AS fecha_ultima_resena
73 FROM usuario u
74 JOIN resena r ON u.id_usuario = r.id_usuario
75 GROUP BY u.id_usuario, u.handle, u.fecha_registro
76 ORDER BY cantidad_resenas_escritas DESC
77 LIMIT 20;
```